QuakeWatch

Revision-Aware Earthquake Intelligence Platform on ArangoDB

Review 2 Report

Database Implementation, Query Evaluation, Integrity Mechanisms and Backend-Database Connectivity with a Verified CRUD Demonstration

Item	Detail
Subject	Database Management Systems (NoSQL database project)
Database	ArangoDB 3.12.5 (multi-model: document, graph, key-value, search)
Backend / Frontend	Python 3.12, Django, Django REST Framework / HTML, CSS, JavaScript, Bootstrap, Leaflet, Chart.js
Data source	U.S. Geological Survey (USGS) earthquake feeds, live ingestion
Repository	https://github.com/abharathreddy751/quake_watch_v2
Report date	11 October 2026

Safety statement: QuakeWatch does not predict earthquakes; its alerts are application notifications and not official emergency warnings. Earthquake data courtesy of the U.S. Geological Survey (USGS).

Contents

Abstract 4

Mapping of the Review 2 deliverables to the ArangoDB stack 4

1. Environment and Architecture 5

    1.1 Runtime environment 5

2. Database Implementation 6

    2.1 Database and collection creation 6

    2.2 Document design 8

    2.3 Graph model 9

    2.4 Sample data and record count 10

    2.5 Indexes 11

    2.6 Views 13

    2.7 Triggers and procedures: NoSQL equivalents 14

    2.8 Database export (dump) 15

3. Complex AQL Queries with Output 16

    3.1 Query 1: Event count and magnitude statistics per region 16

    3.2 Query 2: Dashboard summary with subqueries 17

    3.3 Query 3: Per-network statistics joined to network names 19

    3.4 Query 4: Revisions with large magnitude changes (array index) 21

    3.5 Query 5: Ingestion latency percentiles 23

    3.6 Query 6: Significant events per ISO week 25

    3.7 Query 7: A user's alert rules with match counts (edge traversal) 27

    3.8 Query 8: Matched events through a two-hop graph traversal 29

    3.9 Query 9: Event relationships (OUTBOUND and INBOUND traversal) 30

    3.10 Query 10: Map viewport search (GEO_CONTAINS) 32

    3.11 Query 11: Radius search (GEO_DISTANCE) 34

    3.12 Query 12: Location text search (ArangoSearch view) 36

3.13 Query 13: Ingestion run statistics by kind 38
3.14 Query 14: Depth histogram (bucketed aggregation) 40
3.15 Query 15: Nearby events in space and time 41
3.16 Summary of the query execution 43
4. Backend and Database Connectivity: CRUD Demonstration 45
4.1 Authentication and session mechanism 45
4.2 Backend and database connectivity check 45
4.3 Request sequence and verified results 45
4.4 Findings and repairs made during the work 53
5. Test Cases and Results 54
5.1 Automated regression suite 55
6. Discussion and Limitations 55
References 56

Abstract

QuakeWatch ingests real earthquake observations from the USGS, validates and normalises them, keeps every revision of every event, and exposes search, map, analytics, alerting and research-snapshot functions through a Django REST API. ArangoDB is the only stateful service: it stores domain data, user accounts and sessions, the durable job queue, caches, metrics and audit records. This report documents the Review 2 scope for that stack: the implemented database (collections, 2,670 live earthquake documents, indexes, an ArangoSearch view and a named graph), the integrity mechanisms that play the role of triggers and stored procedures in a NoSQL setting, fifteen complex AQL queries executed against the live data with their outputs, the executed test cases, and a fifteen-request HTTP CRUD demonstration that proves backend-to-database connectivity. Every figure in the report was produced by running the real application; nothing is simulated or mocked.

Mapping of the Review 2 deliverables to the ArangoDB stack

The Review 2 checklist is written with relational terminology. Because QuakeWatch uses a NoSQL multi-model database, each item is satisfied by the closest equivalent construct, as listed below. Where ArangoDB has no direct counterpart (native triggers and stored procedures) this is stated explicitly and the alternative mechanism that actually enforces the behaviour is demonstrated.

Table 1: Review 2 checklist mapped to the implemented NoSQL stack

Review 2 item	ArangoDB / QuakeWatch equivalent	Section
DB dump (.sql)	Native arangodump export (structure + data per collection); JSON schema and index definitions via the repository's idempotent arango_migrate command	2.8
Screenshots of 500+ records	events collection holds 2,670 real documents (2,655 at the time of Figure 6); event_revisions 2,767	2.4
Views	ArangoSearch view events_search (full-text, primary sort on occurred_at) and the named graph quakewatch_graph	2.6
Triggers	Strict JSON-schema validation on write, unique-index constraints, TTL indexes, and ingestion-pipeline observers (no native triggers exist)	2.7
Procedures	Atomic stream transactions (run_in_transaction), a catalogue of 53 named parameterised AQL queries, durable server-side job queue (no stored procedures exist)	2.7
Indexes	32 named secondary indexes (persistent, geo, TTL) plus primary and edge indexes	2.5
15+ complex queries with output	15 AQL queries (aggregation, graph traversal, geo, ArangoSearch, percentiles) with live output and GUI screenshots	3
Test cases	36 executed test cases plus the 936-test regression suite	5
GitHub link + CRUD working, backend and DB connectivity	Repository link, health endpoint, 15 real HTTP requests with database read-back	4

1. Environment and Architecture

1.1 Runtime environment

The application is deployed locally with Docker Compose as four containers that share one image: the ArangoDB 3.12.5 server, the Django web process (gunicorn), a background worker and a scheduler. The worker and scheduler replace Celery: jobs and schedules are documents in ArangoDB, claimed with atomic AQL updates. No relational database, Redis or Celery is used (the Django setting DATABASES is intentionally empty so that any accidental ORM query fails loudly).

Table 2: Runtime components

Component	Technology	Role
Database	ArangoDB 3.12.5 (container, port 8529)	Only stateful service: events, revisions, users, sessions, jobs, cache, metrics, audit
Web	Django + Django REST Framework, gunicorn (port 8000)	Pages and versioned JSON API under /api/v1/
Worker	apps.jobs worker	Executes queued ingestion, enrichment and alert-evaluation jobs
Scheduler	apps.jobs scheduler	Enqueues the USGS feed synchronisation every 300 seconds
Driver	python-arango	Used only inside repository modules and apps/core/arango (enforced by architecture tests)

Access to the ArangoDB web interface was made with the root account; the password field is masked in Figure 1 and no credential is reproduced in this report.

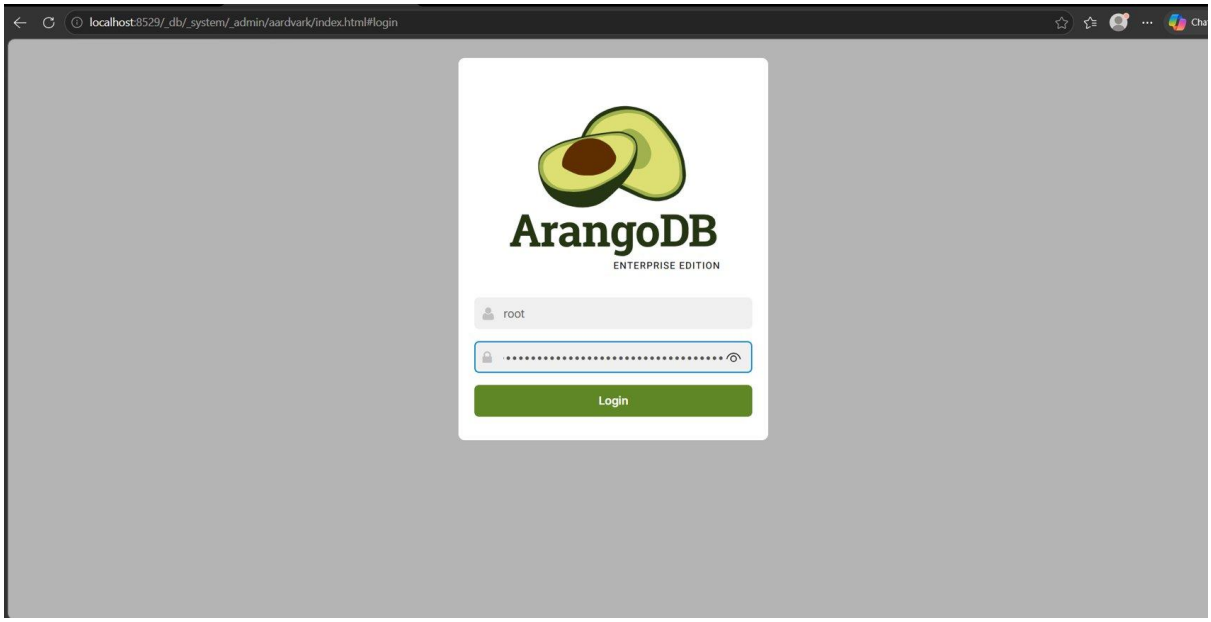


Figure 1: ArangoDB web interface login on localhost:8529 (password masked by the browser).

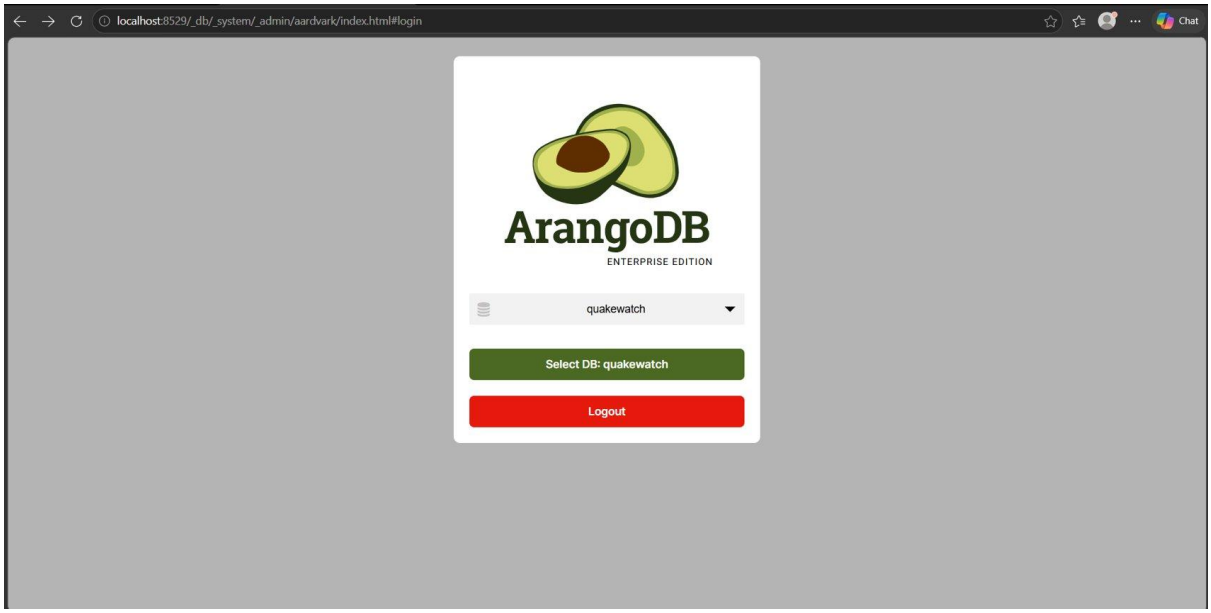


Figure 2: Database selection: the application database quakewatch is chosen after authentication.

2. Database Implementation

2.1 Database and collection creation

The database quakewatch and all of its structure are created by the idempotent management command arango_migrate, which applies schema version 6: collections with JSON-schema validators, persistent, geo and TTL indexes, the ArangoSearch view and the named graph. Re-running the command changes nothing, so the structure is reproducible on any machine. The interface lists 31 collections (Figure 3): 27 document collections and 4 edge collections.



Figure 3: ArangoDB web interface listing the 31 QuakeWatch collections (document and edge types).

Table 3: Collections, live document counts and purpose (total 17,542 documents)

Collection	Type	Documents	Purpose
alert_matches	document	0	Notification records produced when a rule matches an event
alert_rules	document	1	User-defined alert rules (owner-scoped)
audit_log	document	22	Security and administrative audit trail
auth_users	document	2	Account records (password hashes, staff flags)
backfill_jobs	document	2	Historical backfill requests and progress
cache_entries	document	3	Query cache with TTL
circuit_breakers	document	0	Upstream circuit-breaker state
dead_letters	document	0	Records that failed validation, kept for retry
event_identities	document	3,402	Maps every USGS associated id to its canonical event (merge support)
event_in_region	edge	2,670	Edge: event to region
event_reported_by_network	edge	2,670	Edge: event to reporting network
event_revisions	document	2,767	Immutable per-change history of each event (before/after, changed fields)
events	document	2,670	Canonical earthquake documents, one per USGS event
id_sequences	document	1	Atomic integer sequences for ids
ingestion_runs	document	148	One document per feed sync, backfill or enrichment run
job_leases	document	0	Distributed leases with TTL
job_schedules	document	2	Periodic job definitions
jobs	document	98	Durable job queue
metric_counters	document	11	Operational counters
metric_series	document	77	Latency samples for percentile dashboards
networks	document	15	Reporting seismic networks (us, ak, nc, ...)
rate_limits	document	2	API throttle counters (TTL-expired)
raw_payloads	document	2,843	Original USGS payloads kept for provenance (TTL-expired)
regions	document	126	Normalised region dimension (US states, countries)
research_snapshots	document	0	Frozen, checksummed event selections for reproducible research
rule_matched_event	edge	0	Edge: rule to matched event
saved_searches	document	0	Per-user saved explorer filters (capped at 50)
schema_meta	document	1	Applied schema version marker
sessions	document	7	Django session store (TTL-expired)
user_owns_rule	edge	1	Edge: user to alert rule

Collection	Type	Documents	Purpose
users	document	1	Identity vertices for the graph

2.2 Document design

Documents are denormalised where reads dominate and referenced by key where relationships matter. An event is stored once under a deterministic key (usgs_<source id>); relationships to its region and network are edges, and every change is an immutable document in event_revisions. A real, unmodified event document read from the live database is shown below.


```
{
 "_key": "usgs_nc75447861",
 "source": "USGS",
 "source_event_id": "nc75447861",
 "associated_ids": [
  "nc75447861"
 ],
 "event_type": "earthquake",
 "magnitude": 1.67,
 "magnitude_type": "md",
 "place": "9 km N of Hollister, CA",
 "occurred_at": "2026-10-11T16:39:30.720Z",
 "source_updated_at": "2026-10-11T16:43:51.390Z",
 "longitude": -121.415832519531,
 "latitude": 36.9268341064453,
 "depth_km": 8.71000003814697,
 "felt": null,
 "cdi": null,
 "mmi": null,
 "alert": null,
 "status": "automatic",
 "tsunami": false,
 "significance": 43,
 "network": "nc",
 "contributors": [
  "nc"
 ],
 "station_count": 29,
 "min_station_distance_deg": 0.09661,
 "rms_travel_time_residual": 0.07,
 "azimuthal_gap_deg": 53,
 "coordinates": [
  -121.415832519531,
  36.9268341064453
 ],
 "derived": {
  "first_seen_via": "feed",
  "first_seen_latency_s": 330.897,
  "region_key": "california",
  "region_name": "California",
  "region_kind": "us_state",
  "region_method_version": 1
 },
 "first_seen_at": "2026-10-11T16:45:01.617Z",
 "last_seen_at": "2026-10-11T16:45:01.617Z",
 "last_ingested_at": "2026-10-11T16:45:01.617Z",
 "revision_count": 1,
 "payload_hash": "b51fddc4b333d0b0f12dcdefb178b86daf21db3aaf9a722e2c1b47ecfee315e5",
```

```
  "schema_version": 1,
  "revision_policy_version": 1,
  "merged_into": null,
  "last_ingestion_run_id": "431647c2a27341f39db5416a3cf5e89e"
}
```


The events collection carries a strict JSON-schema validator (validation level strict): magnitude is restricted to the range -2 to 11, latitude to -90..90, longitude to -180..180, status to automatic/reviewed/deleted, and timestamps must match an ISO-8601 pattern. The validator rejects non-conforming documents server-side regardless of which client wrote them (Figure 4, Section 2.7).

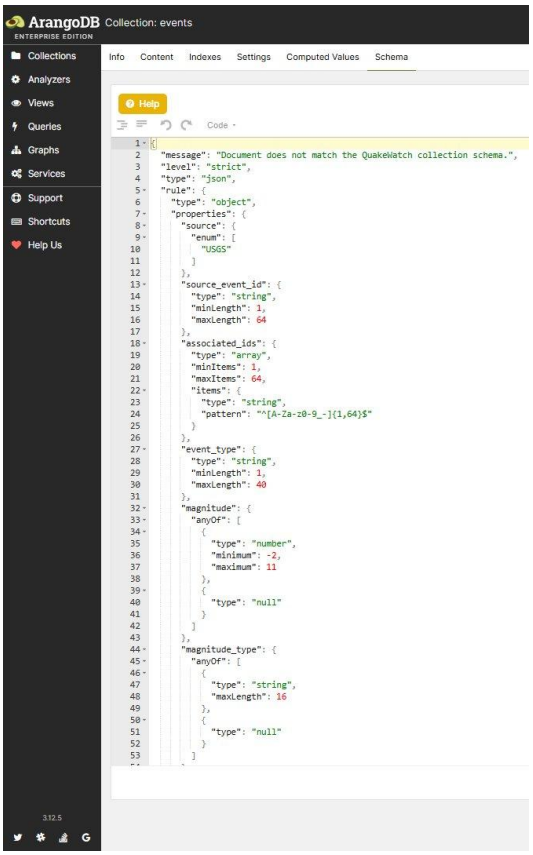


Figure 4: Schema tab of the events collection: server-side JSON-schema validation rules.

2.3 Graph model

The named graph quakewatch_graph connects users to alert rules (user_owns_rule), rules to matched events (rule_matched_event), events to regions (event_in_region) and events to networks (event_reported_by_network). Graph traversals answer questions such as which events matched a user's rules (Query 8) and which events share a region with a given event (Query 9).

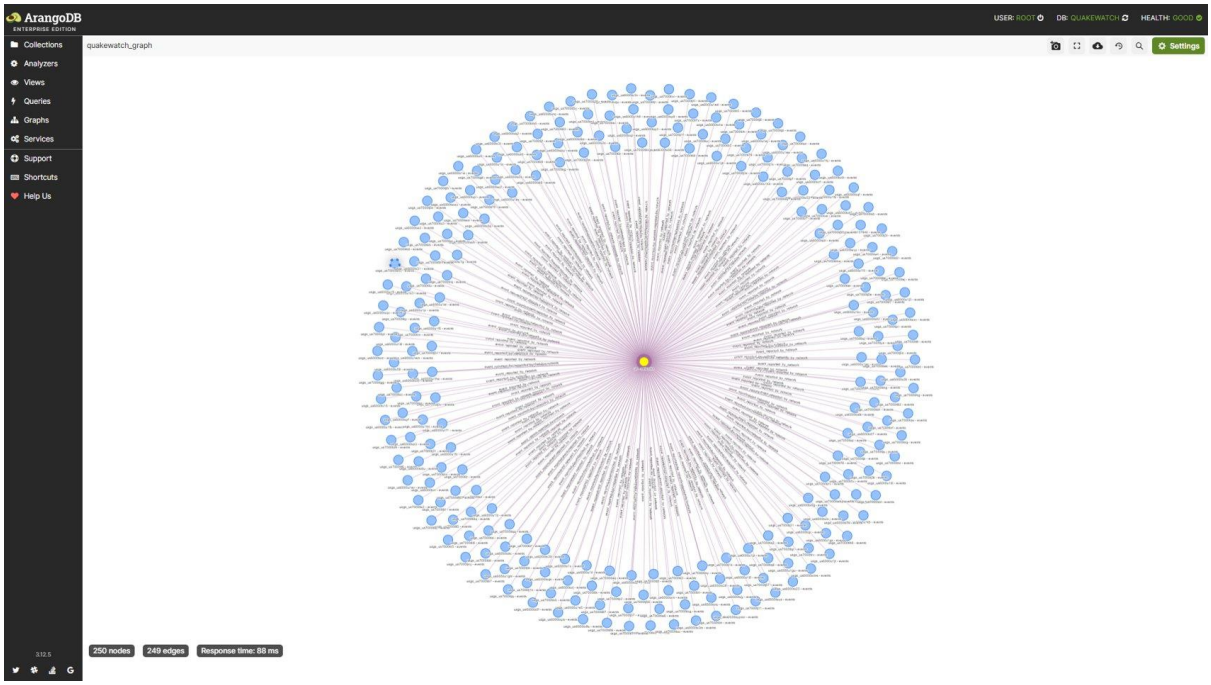


Figure 5: Graph viewer showing quakewatch_graph (250 nodes and 249 edges in the sampled view).

2.4 Sample data and record count

The data are real USGS observations, not generated test data. The events collection contains **2,670 documents** covering 2026-08-26 to 2026-10-11 with magnitudes from -1.05 to 6.6; the revision history holds 2,767 documents and the database as a whole 17,542 documents, which satisfies the requirement of at least 500 records. The count grows as the scheduler ingests new feed data every five minutes, so the screenshot (2,655 documents) was taken slightly earlier than the figures in the tables.

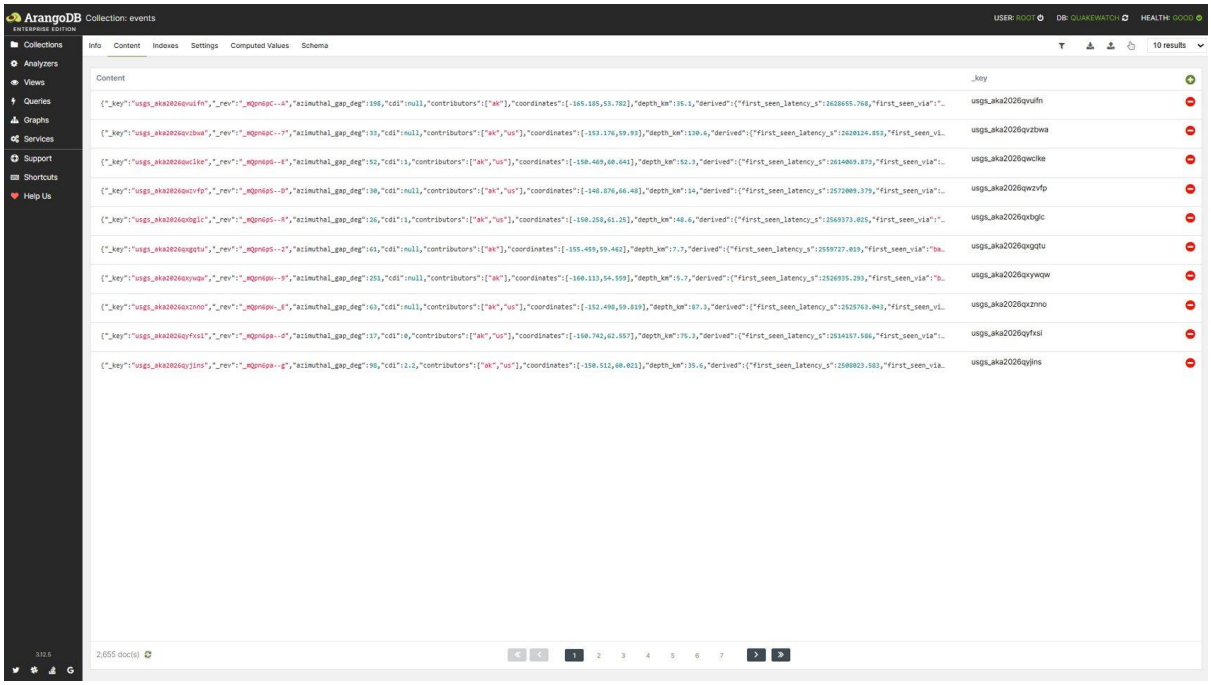


Figure 6: Content tab of the events collection showing 2,655 documents at the time of capture.

Table 4: Record counts of the principal entities (live database)

Entity	Count	Remark
events	2,670	Canonical earthquakes (2026-08-26 to 2026-10-11)
event_revisions	2,767	97 revisions beyond the initial creation records (magnitude, status, location updates)
event_identities	3,402	USGS associated ids mapped to canonical events
raw_payloads	2,843	Provenance copies of source payloads
event_in_region + event_reported_by_network	5,340	2,670 edges each
ingestion_runs	148	138 feed syncs, 8 enrichments, 6 backfill windows
jobs	98	Durable queue entries (TTL-purged when finished)
regions / networks	126 / 15	Dimension collections

2.5 Indexes

Table 5: The 32 named secondary indexes defined by the schema

Index	Collection	Type	Fields	Unique	Purpose
qw_matches_rule_event	alert_matches	persistent	rule_key, event_key	yes	Unique match per (rule, event): no duplicate notifications
qw_matches_user_time	alert_matches	persistent	user_id, matched_at	no	Notification feed per user
qw_rules_user	alert_rules	persistent	user_id	no	Rules of one user
qw_rules_enabled	alert_rules	persistent	enabled	no	Select enabled rules during evaluation
qw_audit_at	audit_log	persistent	at	no	Audit trail by time
qw_auth_username	auth_users	persistent	username	yes	Unique username
qw_auth_username_ci	auth_users	persistent	username_ci	no	Case-insensitive username lookup
qw_backfill_status	backfill_jobs	persistent	status	no	Backfill jobs by status

Index	Collection	Type	Fields	Unique	Purpose
qw_cache_ttl	cache_entries	ttl	expires_at	no	TTL: expire cache entries
qw_dead_last_failure	dead_letters	persistent	last_failure_at	no	Dead letters by last failure
qw_dead_source_event	dead_letters	persistent	source, source_event_id	no	Dead letters per source event
qw_identities_event_key	event_identities	persistent	event_key	no	Resolve all ids of an event
qw_revisions_event_rev	event_revisions	persistent	event_key, revision_number	yes	Unique (event, revision number); history of one event
qw_revisions_observed_at	event_revisions	persistent	observed_at	no	Time-bounded revision analytics
qw_revisions_changed_fields	event_revisions	persistent	changed_fields[*]	no	Array index: find revisions that changed a given field
qw_events_geo	events	geo	coordinates	no	Radius and viewport searches (GEO_DISTANCE, GEO_CONTAINS)
qw_events_source_identity	events	persistent	source, source_event_id, merged_into	yes	Uniqueness of (source, source_event_id, merged_into); idempotent upsert
qw_events_occurred_at	events	persistent	occurred_at, _key	no	Time-ordered explorer pages and analytics windows
qw_events_magnitude	events	persistent	magnitude, _key	no	Magnitude-sorted listings and filters
qw_runs_started_at	ingestion_runs	persistent	started_at	no	Chronological run listing
qw_runs_status	ingestion_runs	persistent	status	no	Filter runs by status
qw_runs_kind_started	ingestion_runs	persistent	kind, started_at	no	Latest runs per kind (operations dashboard)
qw_leases_ttl	job_leases	ttl	expires_at	no	TTL: release abandoned leases
qw_jobs_claim	jobs	persistent	status, run_at	no	Claim the next runnable job
qw_jobs_dedupe	jobs	persistent	dedupe_key	yes	Unique dedupe key: no duplicate jobs
qw_jobs_task_created	jobs	persistent	task, created_at	no	Job history per task
qw_jobs_purge	jobs	ttl	purge_at	no	TTL: remove finished jobs
qw_rate_limits_ttl	rate_limits	ttl	expires_at	no	TTL: expire throttle counters
qw_raw_ttl	raw_payloads	ttl	expires_at	no	TTL: expire raw payload copies
qw_snapshots_user	research_snapshots	persistent	user_id, created_at	no	Snapshots per user
qw_saved_user	saved_searches	persistent	user_id, created_at	no	Saved searches per user
qw_sessions_ttl	sessions	ttl	expire_at	no	TTL: expire sessions

Every named query in the catalogue declares the index it is expected to use, and the diagnostics endpoint runs EXPLAIN against the live optimizer to confirm it. The index actually chosen for each of the fifteen queries of Section 3 is reported with its output.

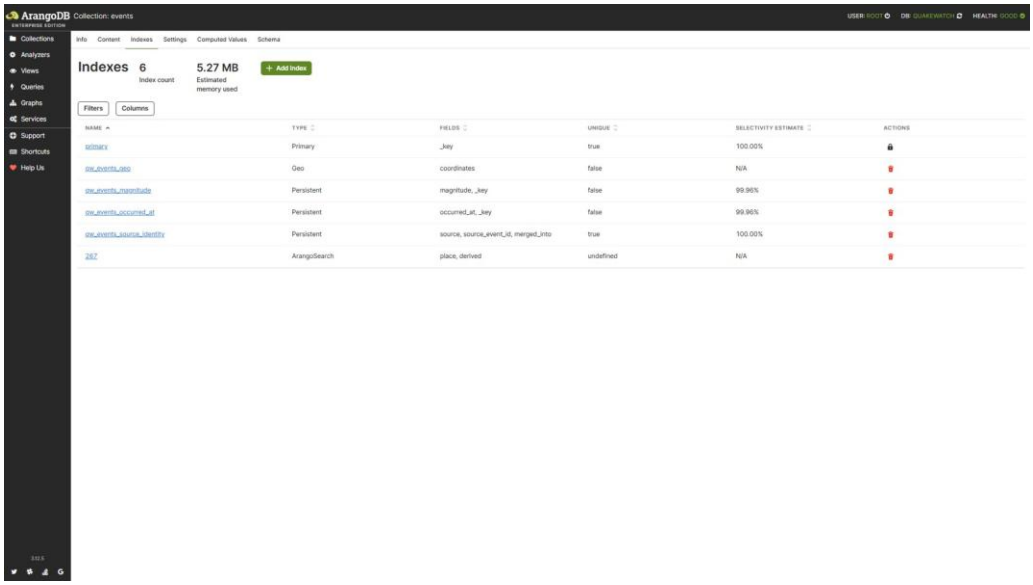


Figure 7: Indexes tab of the events collection: primary, geo, three persistent indexes and the ArangoSearch link.

2.6 Views

ArangoSearch view events_search links the events collection and indexes the place field with the text_en analyzer and derived.region_name with text_en and identity analyzers. Its primary sort is occurred_at descending, so the most recent matches are read first without a separate sort. Query 12 uses the view for tokenised location search.

```
{
 "name": "events_search",
 "type": "arangosearch",
 "links": {
  "events": {
   "analyzers": [
    "identity"
   ],
   "fields": {
    "place": {
     "analyzers": [
      "text_en"
     ]
    },
    "derived": {
     "fields": {
      "region_name": {
       "analyzers": [
        "text_en",
        "identity"
       ]
      }
     }
    }
   },
   "includeAllFields": false,
   "storeValues": "none",
   "trackListPositions": false
  }
 },
```

```
  "primarySort": [
    {
      "field": "occurred_at",
      "asc": false
    }
  ],
  "commitIntervalMsec": 1000
}
```

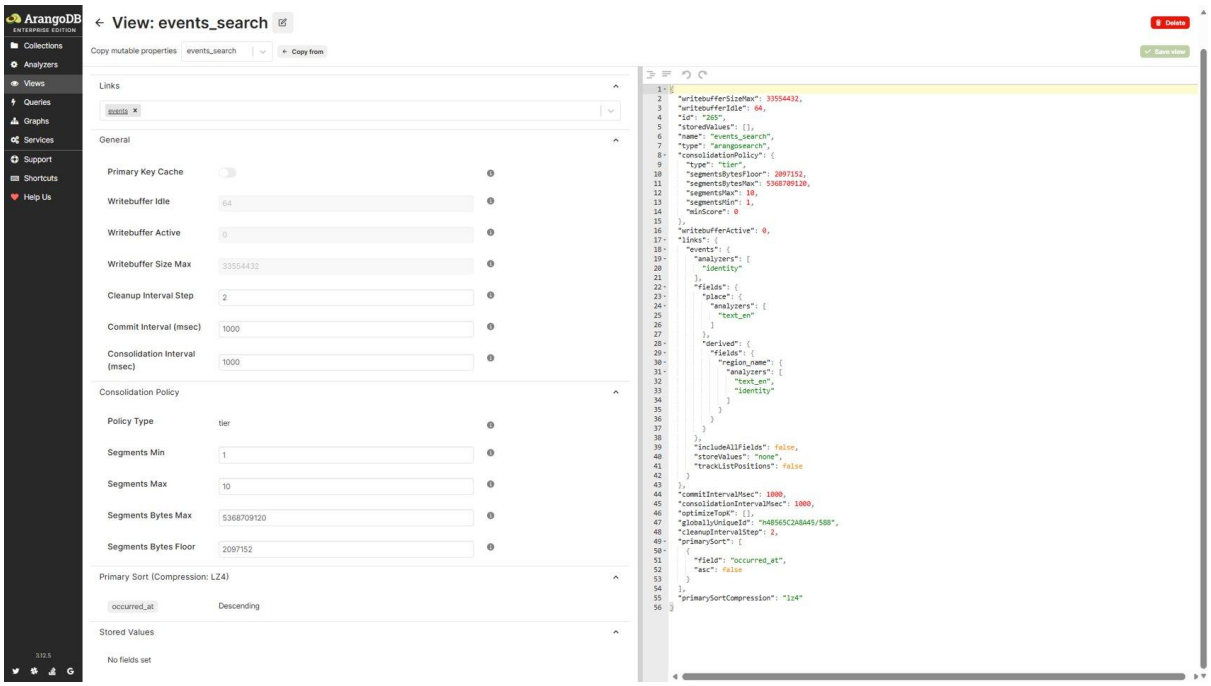


Figure 8: ArangoSearch view events_search: links, analyzers and primary sort.

2.7 Triggers and procedures: NoSQL equivalents

ArangoDB has no SQL-style triggers and no stored procedures; the AQL user-function registry of this database is empty (verified through the HTTP API). The behaviours that triggers and procedures provide in a relational system are obtained here from the following mechanisms, each of which was exercised against the live database.

Table 6: Relational constructs and their ArangoDB counterparts

Relational concept	Mechanism in QuakeWatch	Where
BEFORE INSERT trigger / CHECK constraint	Strict JSON-schema validation on events, alert_rules and other collections; rejected writes fail with error 1620	ArangoDB server
UNIQUE constraint	Unique persistent indexes, e.g. qw_events_source_identity; violations fail with error 1210	ArangoDB server
Scheduled clean-up (event / trigger)	TTL indexes on sessions, cache_entries, rate_limits, raw_payloads, job_leases, jobs	ArangoDB server
AFTER UPDATE trigger	Ingestion pipeline calls registered observers (for example the alert-rule observer) inside the same unit of work after an event changes	apps/ingestion, apps/alerts
Stored procedure	Named parameterised AQL queries (53, registered in apps/core/arango/catalog.py) executed with bind variables only	application

Relational concept	Mechanism in QuakeWatch	Where
BEGIN / COMMIT / ROLLBACK	Stream transactions through run_in_transaction with automatic retry on write-write conflicts (error 1200)	apps/core/arango/transactions.py

The four mechanisms that can be demonstrated directly were executed on the live database with no residual change (document counts of events and audit_log were identical before and after).

Table 7: Integrity mechanisms exercised on the live database

Demonstration	Action	Observed result
Schema validation	Insert an event with source NOT_USGS and magnitude 99	Rejected, error 1620: Document does not match the QuakeWatch collection schema.
Unique index	Insert a copy of an existing event under a new _key	Rejected, error 1210: unique constraint on index qw_events_source_identity
Transaction rollback	In one stream transaction insert a valid audit_log document, then an invalid event	Transaction aborted (error 1620); probe audit document present afterwards: False
Transaction commit	Insert a valid audit_log document in a stream transaction	Committed, document visible: True (removed afterwards)

2.8 Database export (dump)

The native export tool arangodump produces the structure and data of each collection (a structure.json and a gzip-compressed data.json.gz file per collection, a dump.json manifest and the view definition). A dump of an isolated test database was generated on 11 October 2026 and is stored under verification_evidence/review2_2026-10-11/native_arangodump_2026-10-11 (31 collections, 32.8 KB compressed, manifest and all 31 payloads verified readable). The live database can be exported on demand with the command below; no restore test was performed.

```
docker compose exec arangodb arangodump --server.endpoint tcp://127.0.0.1:8529 \
  --server.database quakewatch --output-directory /tmp/qw_dump --overwrite true
```

3. Complex AQL Queries with Output

Fifteen queries were selected from the application's named-query catalogue because together they cover aggregation (COLLECT, AGGREGATE, PERCENTILE), subqueries, joins by DOCUMENT(), array-index lookups, multi-hop graph traversals in both directions, geospatial search, ArangoSearch full-text search and time-window analytics. Each query was executed against the live database with bind variables, and the first rows of the real result are reproduced. The screenshots were captured in the ArangoDB query editor with the same AQL; because the feed is live, counts in the screenshots and in the scripted run can differ by a few documents. For Queries 7 and 8 the screenshots were taken with user_id 1, who owned no rules at that moment (0 rows); the scripted run used a user with one enabled rule so that the traversal returned data.

3.1 Query 1: Event count and magnitude statistics per region

Catalogue name: analytics_avg_magnitude_by_region. **Purpose:** Event count and average/max magnitude by derived region (README Q3). **Techniques:** aggregation.

```
FOR e IN events
  FILTER e.occurred_at >= @since FILTER e.merged_into == null
  COLLECT region = e.derived.region_name
  AGGREGATE n = LENGTH(1), avg_mag = AVERAGE(e.magnitude), max_mag = MAX(e.magnitude)
  SORT n DESC
  LIMIT @limit
  RETURN {region: region, count: n, avg_magnitude: ROUND(avg_mag * 100) / 100,
          max_magnitude: max_mag}
```

Bind variables: {"since": "2026-08-26T00:00:00.000Z", "limit": 20}


```
[
  {
    "region": "Alaska",
    "count": 981,
    "avg_magnitude": 2.82,
    "max_magnitude": 6.5
  },
  {
    "region": "California",
    "count": 353,
    "avg_magnitude": 1.41,
    "max_magnitude": 4.74
  },
  {
    "region": "Indonesia",
    "count": 135,
    "avg_magnitude": 4.6,
    "max  magnitude": 6.5
  }
]
```


Result: 20 row(s) returned in 6.98 ms. **Index chosen by the optimizer:** qw_events_occurred_at.

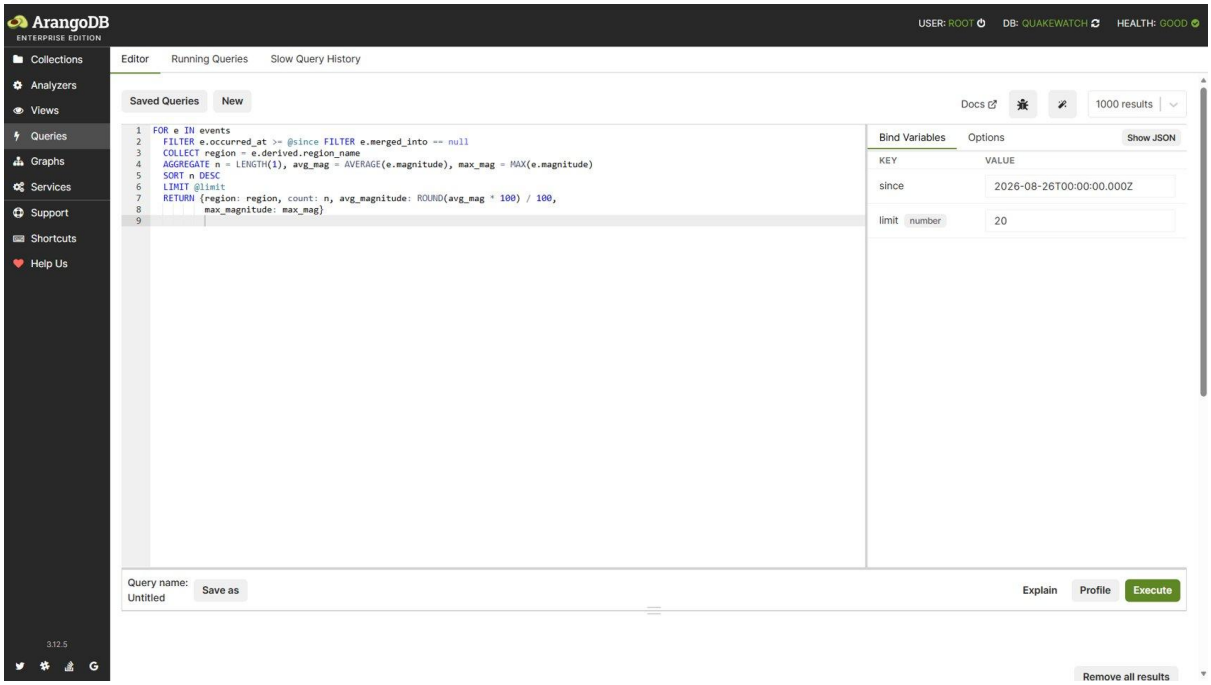


Figure 9: Query 1: Query editor with AQL and bind variables (since, limit).

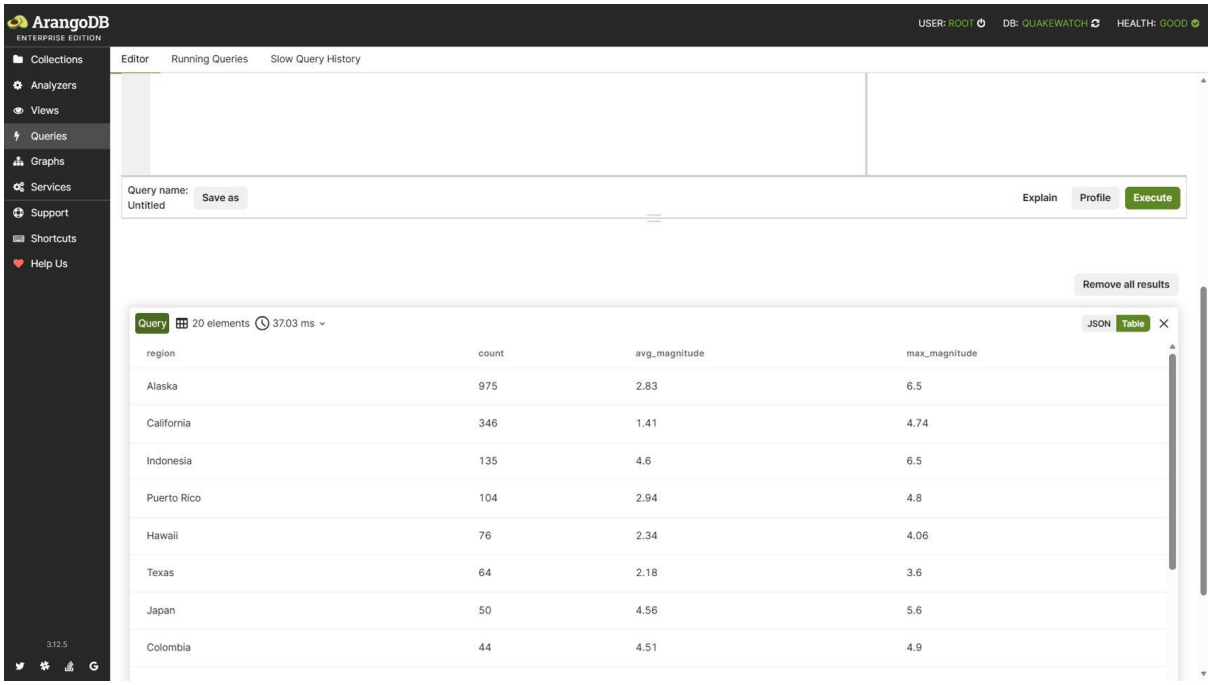


Figure 10: Query 1: Result table: 20 regions ranked by event count.

3.2 Query 2: Dashboard summary with subqueries

Catalogue name: analytics_dashboard_summary. **Purpose:** Situational summary: counts in the last 24 h / 7 d, strongest events, significant events. **Techniques:** aggregation, subqueries.

```
LET day = (FOR e IN events FILTER e.occurred_at >= @day FILTER e.merged_into == null
        RETURN e)
LET week = (FOR e IN events FILTER e.occurred  at >= @week FILTER e.merged  into == null
         RETURN KEEP(e, "_key", "magnitude", "place", "occurred_at", "alert",
```

```
                    "significance", "tsunami", "depth_km", "status"))
RETURN {
  count_24h: LENGTH(day),
  count_7d: LENGTH(week),
  m45_plus_7d: LENGTH(FOR e IN week FILTER e.magnitude >= 4.5 RETURN 1),
  strongest_24h: FIRST(FOR e IN day FILTER e.magnitude != null SORT e.magnitude DESC LIMIT 1
                    RETURN MERGE(KEEP(e, "magnitude", "place", "occurred_at", "depth_km"),
                                 {key: e._key})),
  strongest_7d: FIRST(FOR e IN week FILTER e.magnitude != null SORT e.magnitude DESC LIMIT 1
                   RETURN MERGE(UNSET(e, "_key"), {key: e._key})),
  significant_7d: (FOR e IN week
                   FILTER e.significance >= 600 OR e.alert != null OR e.tsunami
                   SORT e.significance DESC LIMIT 10
                   RETURN MERGE(UNSET(e, "_key"), {key: e._key})),
  reviewed_share_7d: LENGTH(week) == 0 ? null
    : LENGTH(FOR e IN week FILTER e.status == "reviewed" RETURN 1) / LENGTH(week)
}
```

Bind variables: {"day": "2026-08-26T00:00:00.000Z", "week": "2026-08-26T00:00:00.000Z"}


```
[
  {
    "count_24h": 2671,
    "count_7d": 2671,
    "m45_plus_7d": 553,
    "strongest_24h": {
      "depth_km": 10,
      "magnitude": 6.6,
      "occurred_at": "2026-09-25T21:23:03.309Z",
      "place": "80 km ENE of Tadine, New Caledonia",
      "key": "usgs_us6000txpi"
    },
    "strongest_7d": {
      "alert": "green",
      "depth_km": 10,
      "magnitude": 6.6,
      "occurred_at": "2026-09-25T21:23:03.309Z",
      "place": "80 km ENE of Tadine, New Caledonia",
      "significance": 678,
      "status": "reviewed",
      "tsunami": false,
      "key": "usgs_us6000txpi"
    },
    "significant_7d": [
      {
        "alert": "green",
        "depth_km": 10,
        "magnitude": 6.6,
        "occurred_at": "2026-09-25T21:23:03.309Z",
        "place": "80 km ENE of Tadine, New Caledonia",
        "significance": 678,
        "status": "reviewed",
        "tsunami": false,
        "key": "usgs_us6000txpi"
      },
      {
        "alert": "green",
        "depth_km": 372,
        "magnitude": 6.5,
        "occurred_at": "2026-09-11T21:23:55.907Z",
  ... (truncated for the report)
```


Result: 1 row(s) returned in 32.24 ms. **Index chosen by the optimizer:** qw_events_occurred_at.

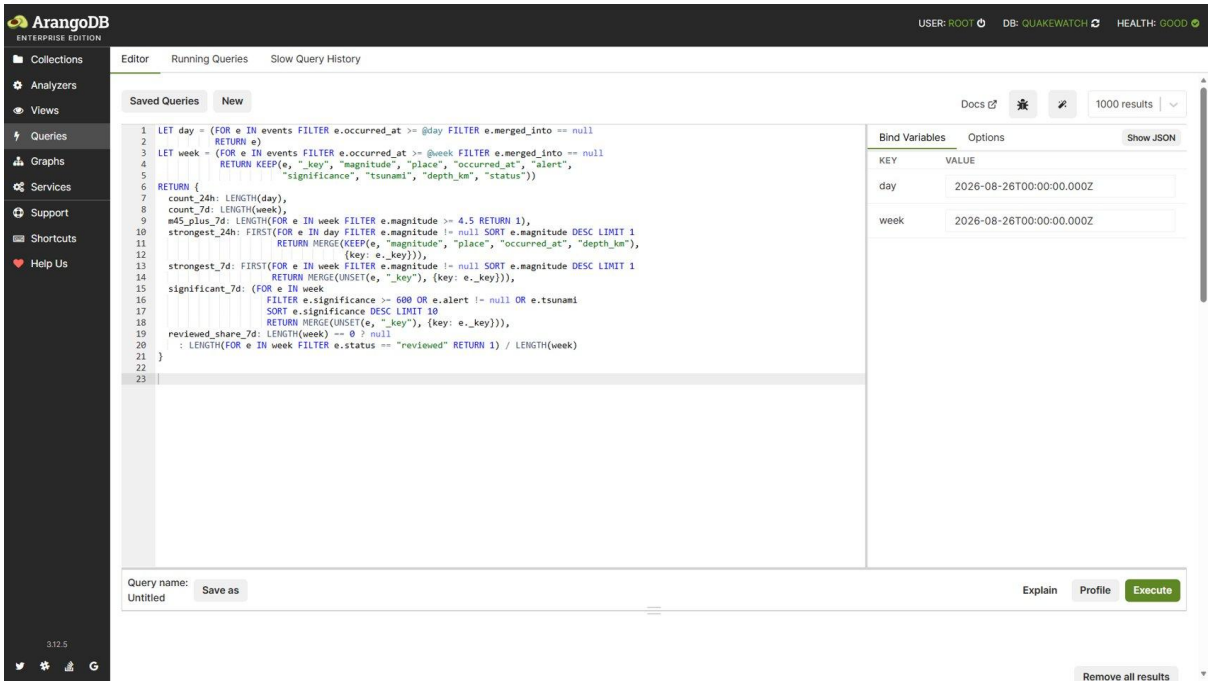


Figure 11: Query 2: Query editor with the dashboard summary AQL.

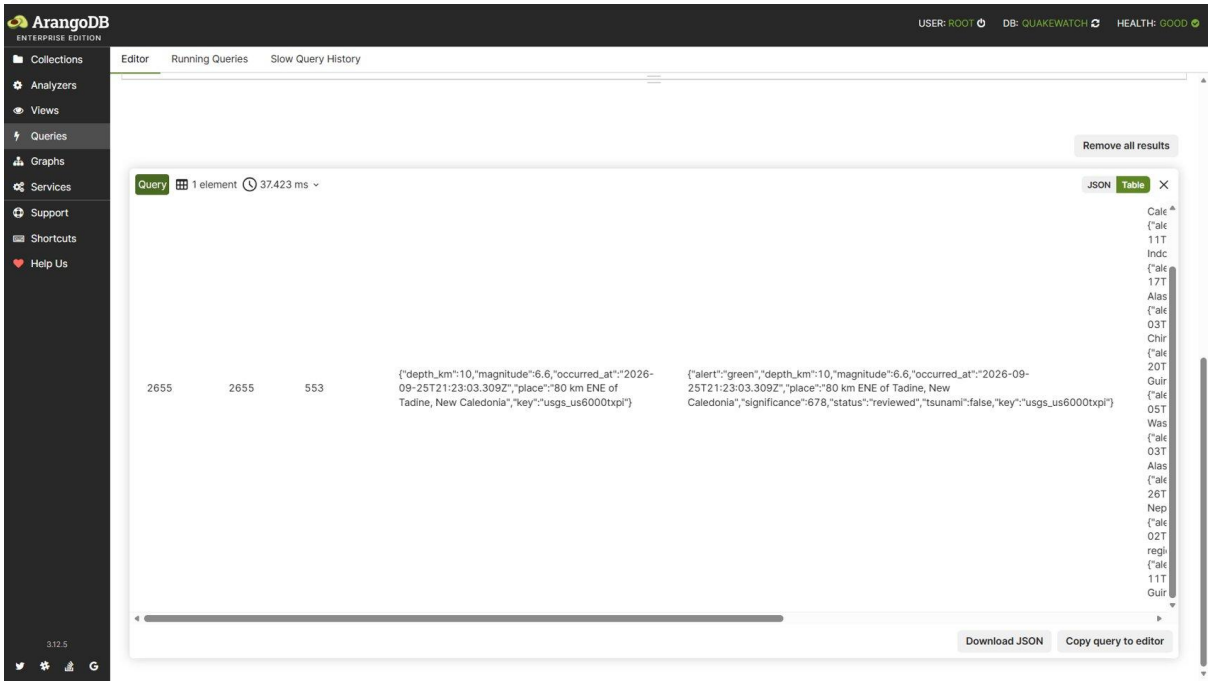


Figure 12: Query 2: Result: counts, strongest event and significant events.

3.3 Query 3: Per-network statistics joined to network names

Catalogue name: analytics_network_stats. **Purpose:** Per-network event statistics joined with network names via the graph (README Q11). **Techniques:** aggregation, join.

```
FOR e IN events
  FILTER e.occurred_at >= @since FILTER e.merged_into == null
  COLLECT code = e.network
  AGGREGATE n = LENGTH(1), avg_mag = AVERAGE(e.magnitude), max_mag = MAX(e.magnitude),
```

```
        reviewed = SUM(e.status == "reviewed" ? 1 : 0)
  LET network = DOCUMENT("networks", code)
  SORT n DESC
  RETURN {network: code, name: network.name, count: n,
        avg_magnitude: ROUND(avg_mag * 100) / 100, max_magnitude: max_mag,
        reviewed_share: ROUND(reviewed / n * 1000) / 1000}
```

Bind variables: {"since": "2026-08-26T00:00:00.000Z"}


```
[
  {
    "network": "us",
    "name": "USGS National Earthquake Information Center",
    "count": 1295,
    "avg_magnitude": 4.16,
    "max_magnitude": 6.6,
    "reviewed_share": 1
  },
  {
    "network": "ak",
    "name": "Alaska Earthquake Center",
    "count": 617,
    "avg_magnitude": 2.66,
    "max_magnitude": 4.8,
    "reviewed_share": 0.749
  },
  {
    "network": "nc",
    "name": "Northern California Seismic System",
    "count": 191,
    "avg_magnitude": 1.49,
    "max_magnitude": 4.74,
    "reviewed_share": 0.126
  }
]
```


Result: 15 row(s) returned in 6.83 ms. **Index chosen by the optimizer:** qw_events_occurred_at.

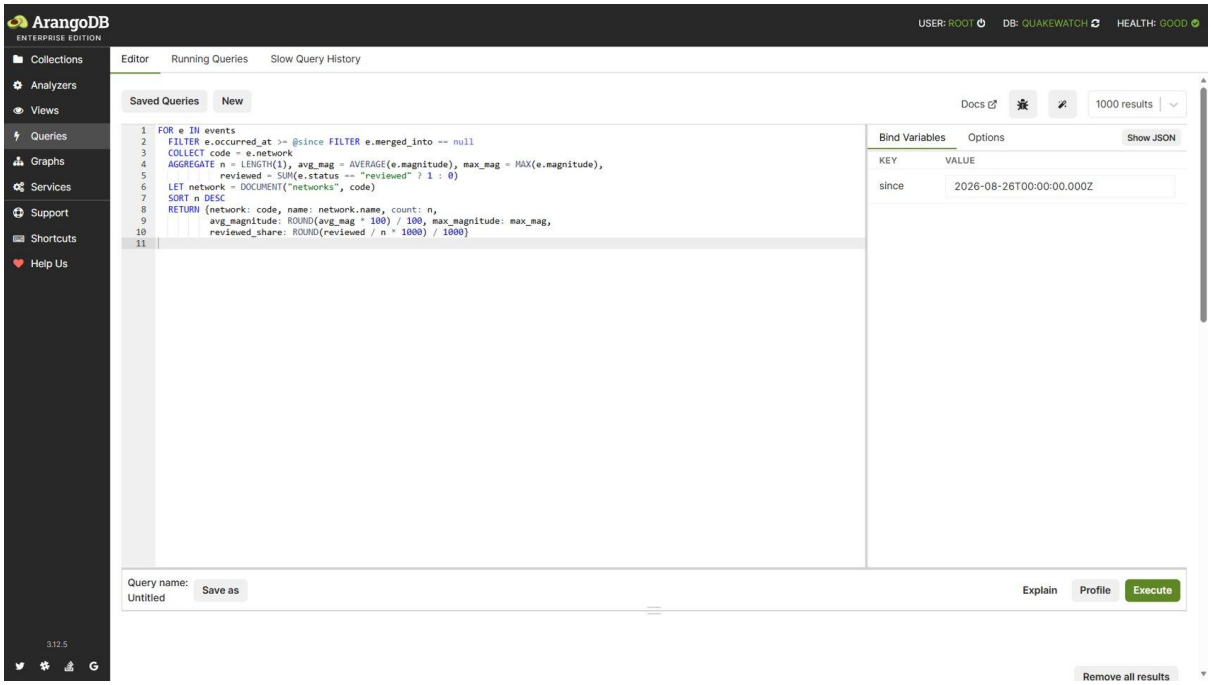


Figure 13: Query 3: Query editor with the network statistics AQL.

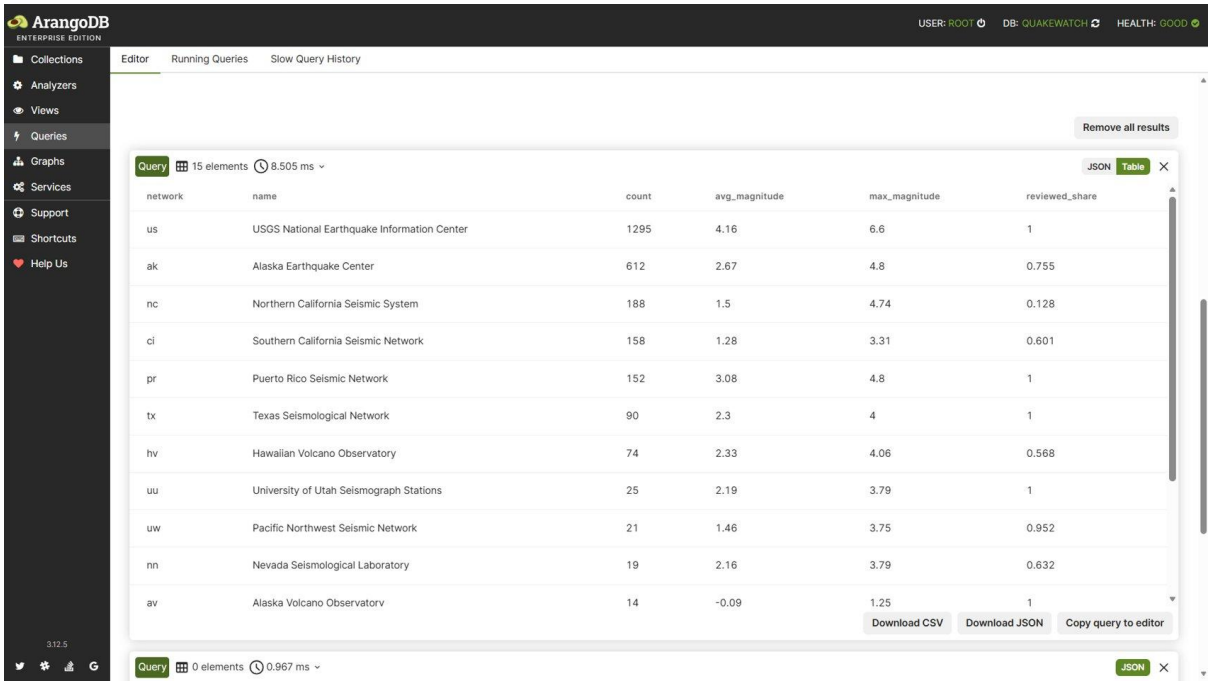


Figure 14: Query 3: Result table: 15 networks with names joined by DOCUMENT().

3.4 Query 4: Revisions with large magnitude changes (array index)

Catalogue name: analytics_magnitude_changes. **Purpose:** Revisions whose magnitude changed by at least a threshold (README Q14). **Techniques:** array index, join.

```
FOR r IN event_revisions
  FILTER "magnitude" IN r.changed_fields[*]
  FILTER r.before.magnitude != null AND r.after.magnitude != null
  LET delta = r.after.magnitude - r.before.magnitude
```

```
  FILTER ABS(delta) >= @threshold
  SORT ABS(delta) DESC, r.observed_at DESC
  LIMIT @limit
  LET e = DOCUMENT("events", r.event_key)
  RETURN {key: r.event_key, revision_number: r.revision_number, observed_at: r.observed_at,
          before: r.before.magnitude, after: r.after.magnitude,
          delta: ROUND(delta * 100) / 100, place: e.place}
```

Bind variables: {"threshold": 0.3, "limit": 20}


```
[
  {
    "key": "usgs_ci41339079",
    "revision_number": 2,
    "observed_at": "2026-09-26T11:25:00.784Z",
    "before": 1.33521321207945,
    "after": 0.26,
    "delta": -1.08,
    "place": "11 km SSW of Idyllwild, CA"
  },
  {
    "key": "usgs_aka2026udvdhv",
    "revision_number": 2,
    "observed_at": "2026-10-11T14:35:01.152Z",
    "before": 1.3,
    "after": 2,
    "delta": 0.7,
    "place": "18 km N of Beluga, Alaska"
  },
  {
    "key": "usgs_ci41338943",
    "revision_number": 2,
    "observed_at": "2026-09-25T17:22:14.930Z",
    "before": 1.28,
    "after": 0.64,
    "delta": -0.64,
    "place": "4 km WSW of Anza, CA"
  }
]
```


Result: 10 row(s) returned in 3.15 ms. **Index chosen by the optimizer:** qw_revisions_changed_fields.

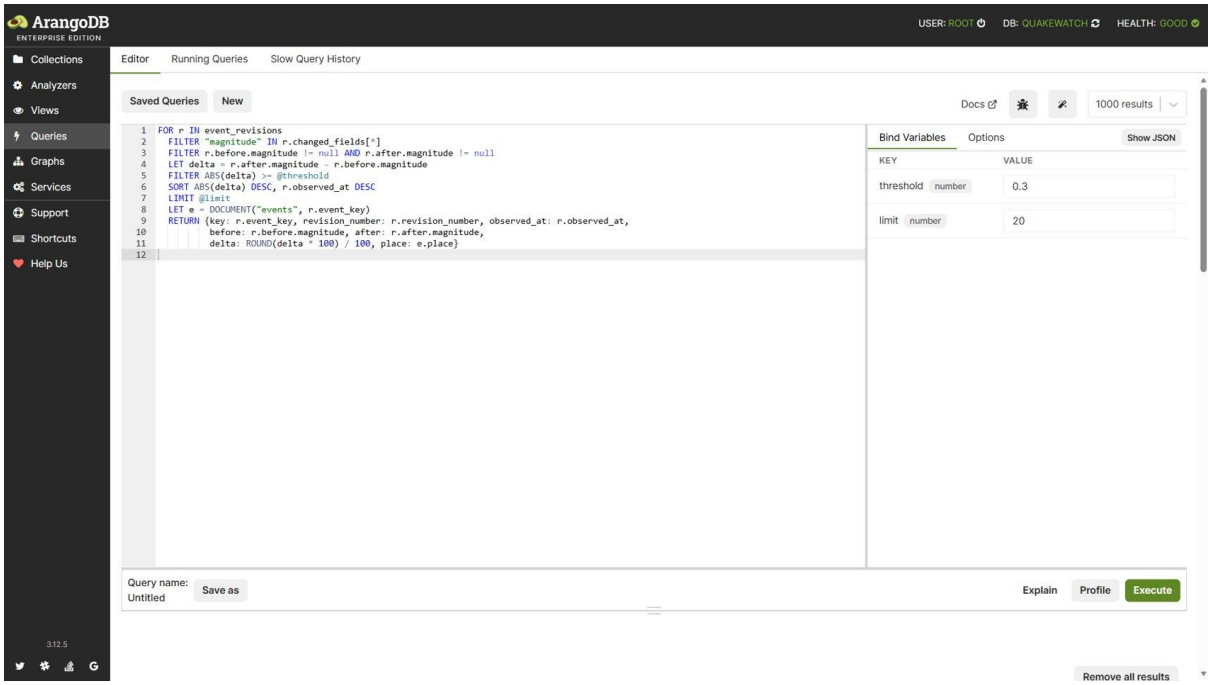


Figure 15: Query 4: Query editor with the revision-delta AQL (threshold 0.3).

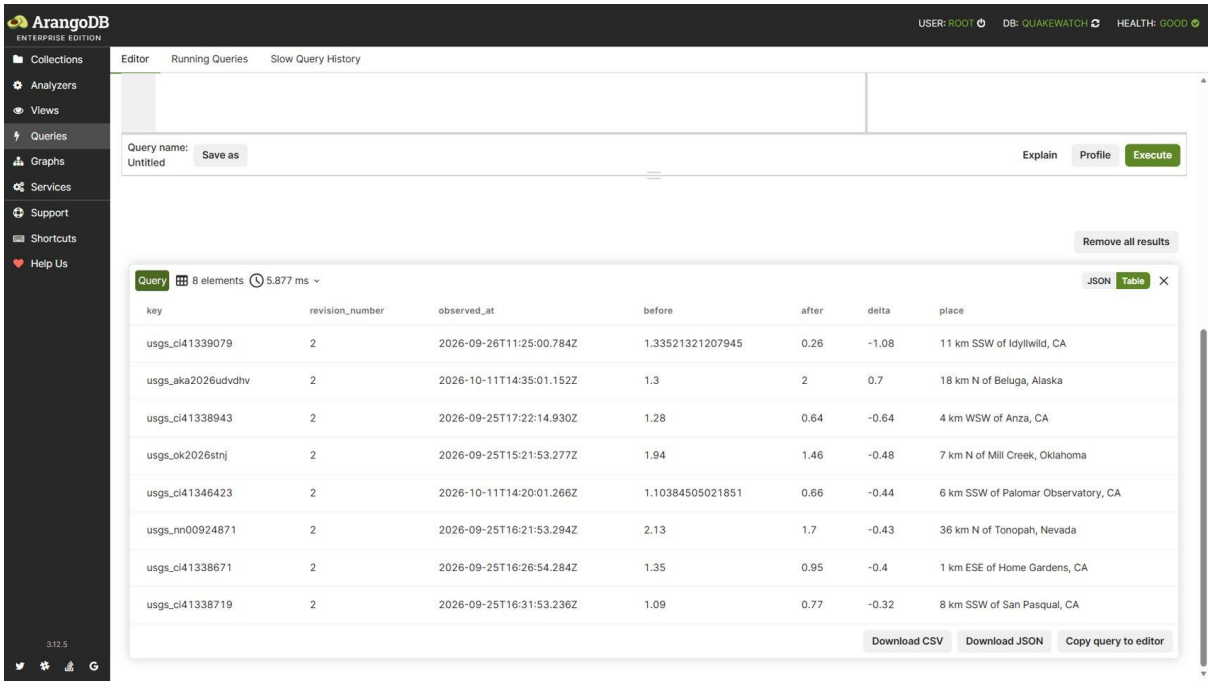


Figure 16: Query 4: Result: eight revisions whose magnitude changed by at least 0.3.

3.5 Query 5: Ingestion latency percentiles

Catalogue name: analytics_ingestion_latency. **Purpose:** Seconds from origin time to first observation by QuakeWatch, for events that occurred after live monitoring began (README Q15, derived value). **Techniques:** percentiles, subqueries.

```
LET monitoring_start = FIRST(
  FOR r IN ingestion_runs OPTIONS {indexHint: "qw_runs_kind_started"}
```

```
    FILTER r.kind == "feed_sync" AND r.status IN ["succeeded", "partial"]
    SORT r.kind ASC, r.started_at ASC
    LIMIT 1
    RETURN r.started_at
)
LET since = monitoring_start == null ? @since : MAX([@since, monitoring_start])
LET latencies = (
  FOR e IN events
    FILTER e.occurred_at >= since FILTER e.merged_into == null
    FILTER e.derived.first_seen_via == "feed" AND e.derived.first_seen_latency_s != null
    RETURN e.derived.first_seen_latency_s
)
RETURN {
  monitoring_start: monitoring_start,
  count: LENGTH(latencies),
  p50_s: LENGTH(latencies) == 0 ? null : PERCENTILE(latencies, 50),
  p90_s: LENGTH(latencies) == 0 ? null : PERCENTILE(latencies, 90),
  p99_s: LENGTH(latencies) == 0 ? null : PERCENTILE(latencies, 99),
  min_s: MIN(latencies), max_s: MAX(latencies)
}
```

Bind variables: {"since": "2026-08-26T00:00:00.000Z"}


```
[
  {
    "monitoring_start": "2026-09-25T10:42:59.319Z",
    "count": 442,
    "p50_s": 26050.16,
    "p90_s": 69140.77,
    "p99_s": 84195.002,
    "min_s": 120.047,
    "max_s": 85659.403
  }
]
```


Result: 1 row(s) returned in 3.29 ms. **Index chosen by the optimizer:** qw_events_occurred_at, qw_runs_kind_started.

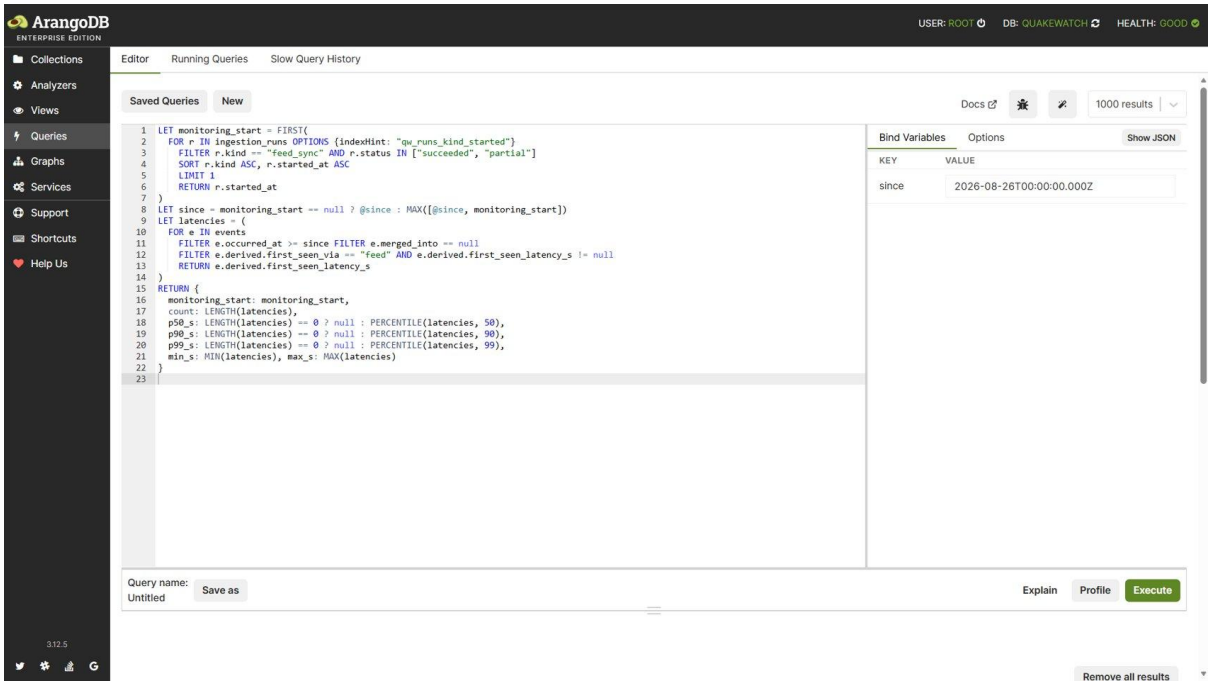


Figure 17: Query 5: Query editor with the latency percentile AQL.

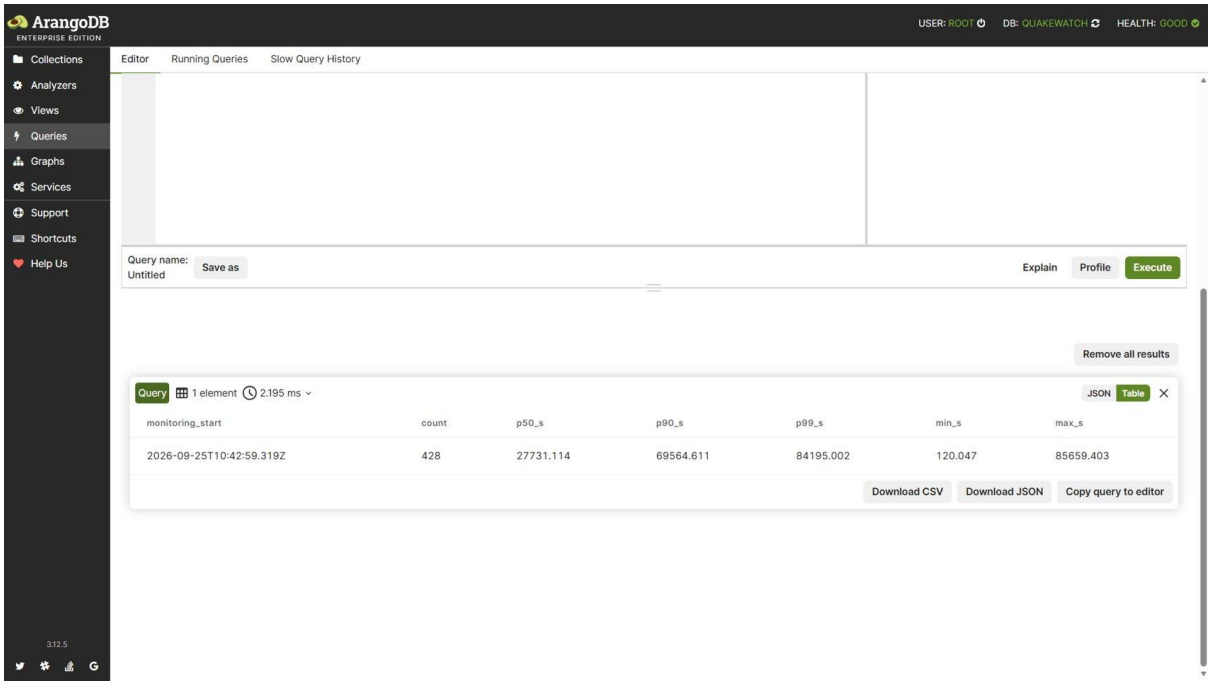


Figure 18: Query 5: Result: first-seen latency percentiles.

3.6 Query 6: Significant events per ISO week

Catalogue name: analytics_significant_by_week. **Purpose:** Significant events (significance ≥ 600, PAGER alert or tsunami flag) per ISO week (README Q12). **Techniques:** aggregation.

```
FOR e IN events
  FILTER e.occurred_at >= @since FILTER e.merged_into == null
```

```
  FILTER e.significance >= 600 OR e.alert != null OR e.tsunami
  LET iso = DATE_ISOWEEKYEAR(e.occurred_at)
  COLLECT week = CONCAT(iso.year, "-W", RIGHT(CONCAT("0", iso.week), 2)) INTO members = e
  SORT week
  RETURN {week: week, count: LENGTH(members),
          strongest: MAX(members[*].magnitude)}
```

Bind variables: {"since": "2026-08-26T00:00:00.000Z"}


```
[
  {
    "week": "2026-W35",
    "count": 5,
    "strongest": 5.9
  },
  {
    "week": "2026-W36",
    "count": 24,
    "strongest": 6.3
  },
  {
    "week": "2026-W37",
    "count": 8,
    "strongest": 6.5
  }
]
```


Result: 6 row(s) returned in 9.78 ms. **Index chosen by the optimizer:** qw_events_occurred_at.

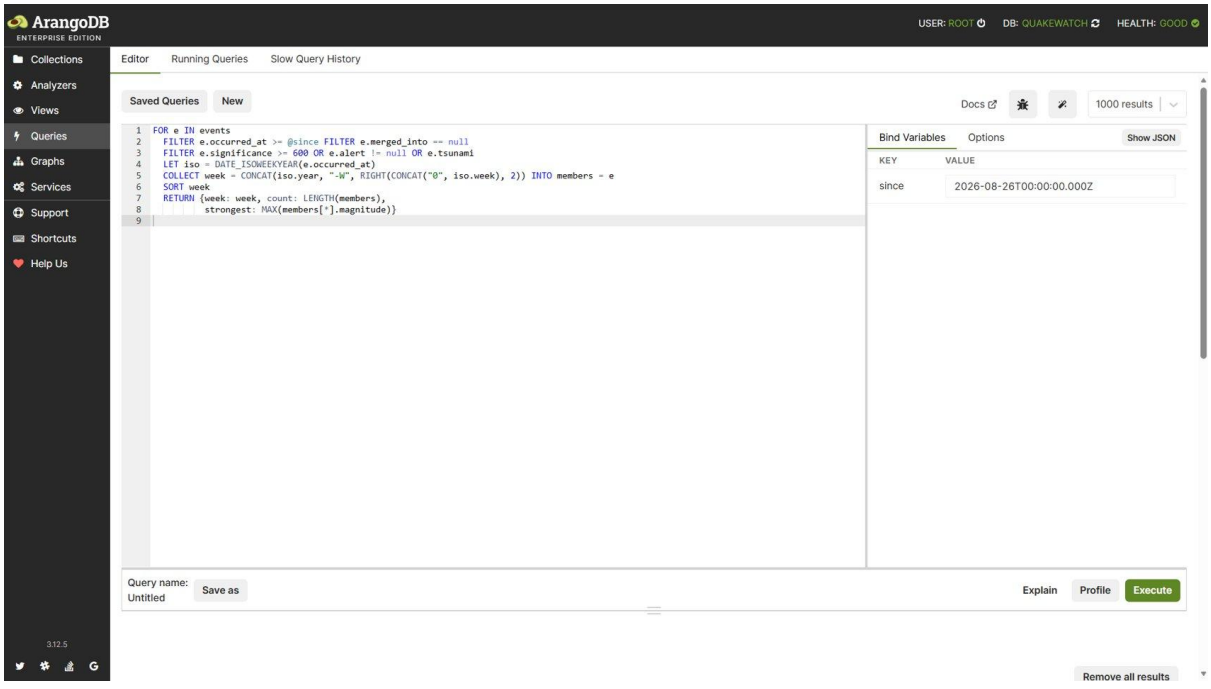


Figure 19: Query 6: Query editor with the ISO-week AQL.

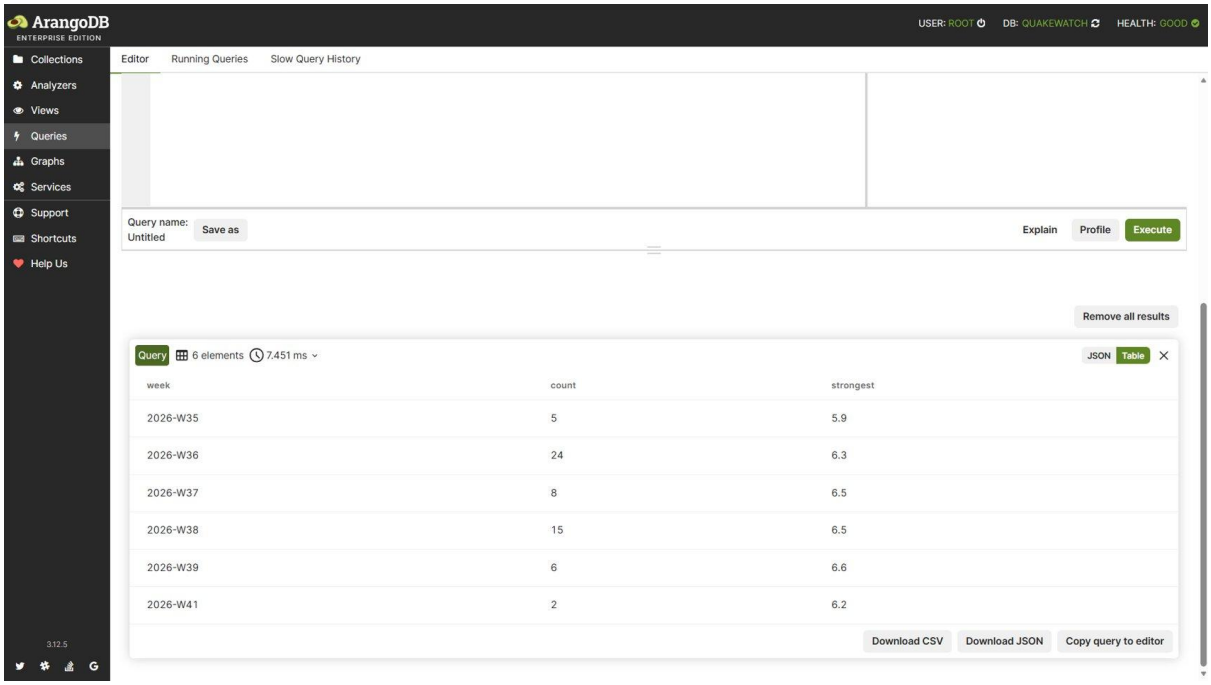


Figure 20: Query 6: Result: six weeks with counts and strongest magnitude.

3.7 Query 7: A user's alert rules with match counts (edge traversal)

Catalogue name: alerts_user_rules. **Purpose:** A user's alert rules with match counts (CRUD list). **Techniques:** persistent index, graph traversal.

```
FOR r IN alert_rules
  FILTER r.user_id == @user_id
  SORT r.created_at DESC
  LET matches = LENGTH(FOR v IN 1..1 OUTBOUND r rule_matched_event RETURN 1)
  RETURN MERGE(UNSET(r, "_id", "_rev", "_key"), {id: r._key, match_count: matches})
```

Bind variables: {"user_id": 2}


```
[
 {
  "conditions": {
   "minimum_magnitude": 1,
   "maximum_depth_km": null,
   "minimum_depth_km": null,
   "center": null,
   "radius_km": null,
   "alert_levels": [],
   "tsunami_only": false,
   "event_types": [
    "earthquake"
   ]
  },
  "cooldown_minutes": 0,
  "created_at": "2026-10-11T16:49:28.349Z",
  "enabled": true,
  "name": "Global M1.0+ evidence rule",
  "updated_at": "2026-10-11T16:49:28.349Z",
  "user_id": 2,
  "match_count": 1,
  "id": "baa80123a8d144709a4d91f1a3996973"
```

```
  }
]
```

Result: 1 row(s) returned in 2.18 ms. **Index chosen by the optimizer:** edge, qw_rules_user.

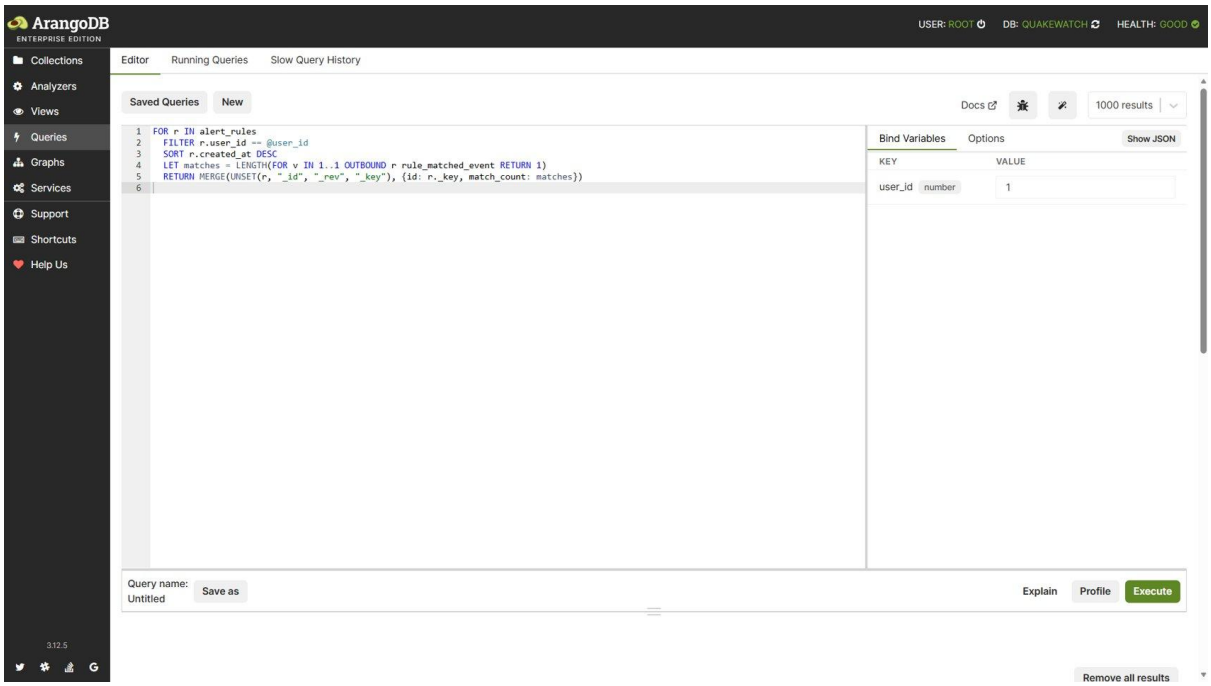


Figure 21: Query 7: Query editor with the user-rules AQL (user_id 1).

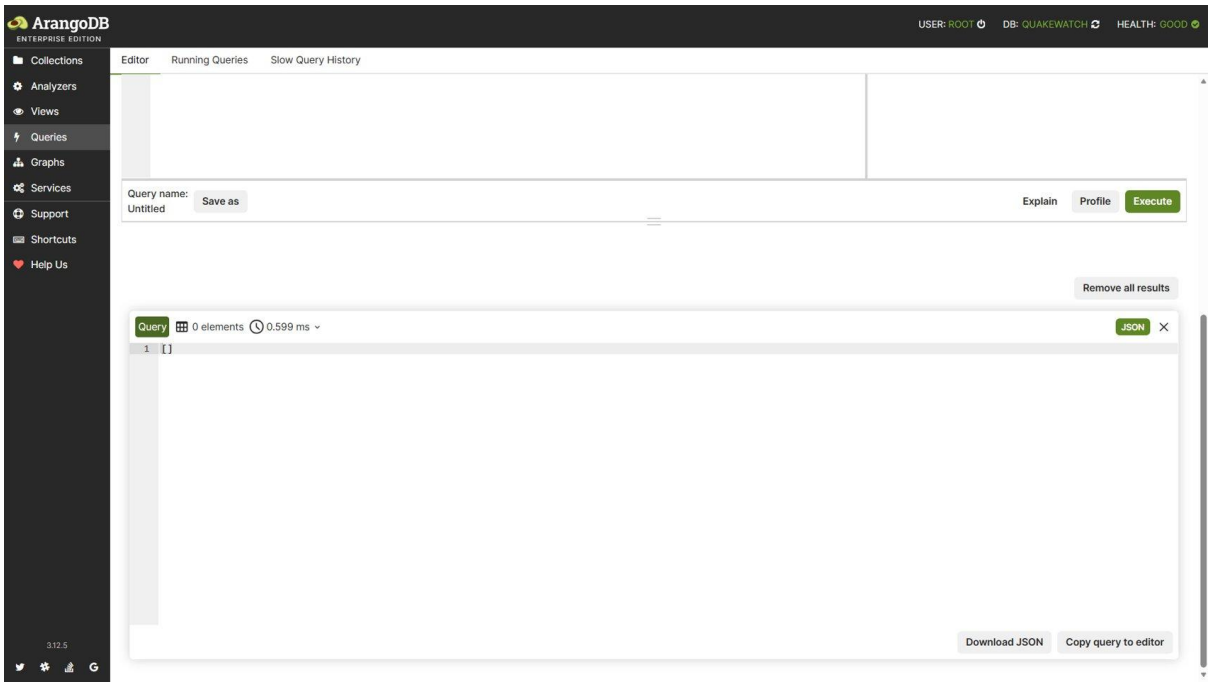


Figure 22: Query 7: Result: empty array, the user owned no rules at that moment.

3.8 Query 8: Matched events through a two-hop graph traversal

Catalogue name: alerts_matched_events_graph. **Purpose:** Events matched by any of a user's watch rules via 2-hop traversal users → rules → events (README Q13). **Techniques:** graph traversal, 2 hops.

```
  FOR rule IN 1..1 OUTBOUND CONCAT("users/", @user_id) user_owns_rule
    FOR ev, edge IN 1..1 OUTBOUND rule rule_matched_event
      FILTER ev.merged_into == null
      SORT edge.matched_at DESC
      LIMIT @limit
      RETURN {rule: rule.name, rule_id: rule._key, event_key: ev._key,
              magnitude: ev.magnitude, place: ev.place, occurred_at: ev.occurred_at,
              matched_at: edge.matched_at}
```

Bind variables: {"user_id": 2, "limit": 20}


```
[
  {
    "rule": "Global M1.0+ evidence rule",
    "rule_id": "baa80123a8d144709a4d91f1a3996973",
    "event_key": "usgs_aka2026udzskb",
    "magnitude": 2.4,
    "place": "22 km NNW of Nikiski, Alaska",
    "occurred_at": "2026-10-11T16:45:11.678Z",
    "matched_at": "2026-10-11T16:50:01.966Z"
  }
]
```


Result: 1 row(s) returned in 1.22 ms. **Index chosen by the optimizer:** edge.

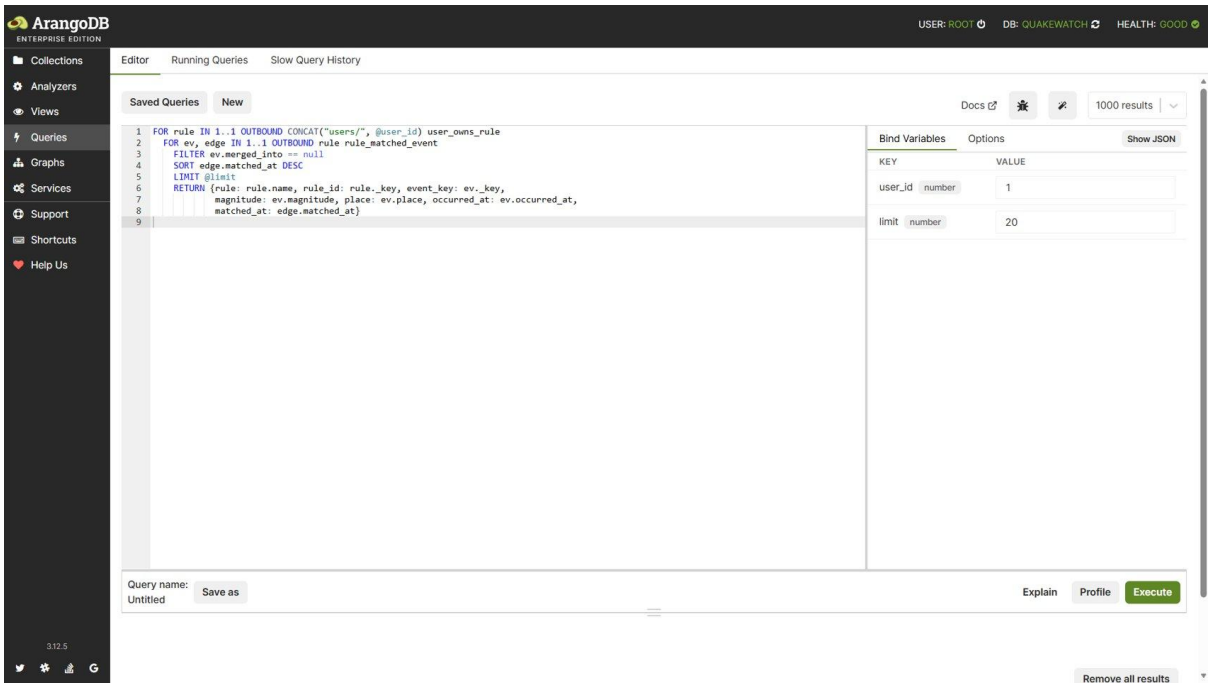


Figure 23: Query 8: Query editor with the two-hop traversal AQL.

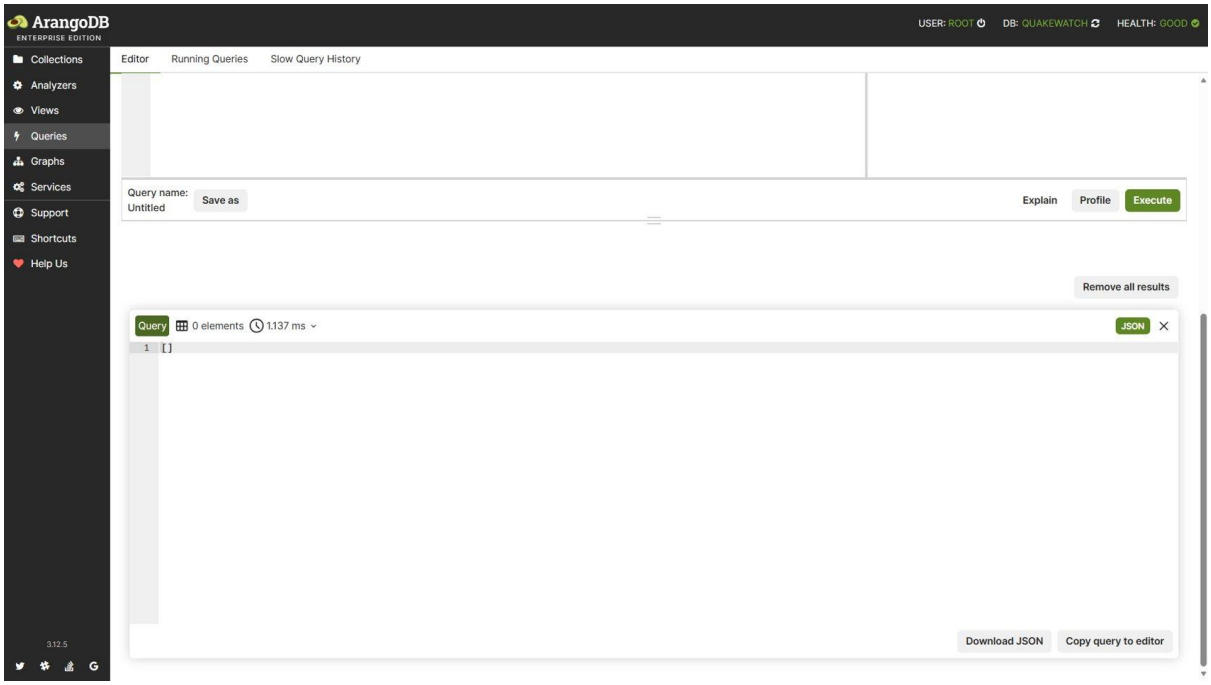


Figure 24: Query 8: Result: empty array for the same user.

3.9 Query 9: Event relationships (OUTBOUND and INBOUND traversal)

Catalogue name: event_graph_relationships. **Purpose:** Region and network of an event and other recent events in the same region (graph traversal event → region ← events; README Q10). **Techniques:** graph traversal, OUTBOUND, INBOUND.

```
  LET start = DOCUMENT("events", @key)
  LET region = FIRST(FOR v IN 1..1 OUTBOUND start event_in_region RETURN v)
  LET network = FIRST(FOR v IN 1..1 OUTBOUND start event_reported_by_network RETURN v)
  LET same_region = region == null ? [] : (
    FOR other IN 1..1 INBOUND region event_in_region
      FILTER other._key != @key AND other.merged_into == null
      SORT other.occurred_at DESC
      LIMIT @limit
      RETURN MERGE(KEEP(other, "magnitude", "place", "occurred_at", "depth_km"),
                {key: other._key})
  )
  LET region_total = region == null ? 0 : LENGTH(
    FOR other IN 1..1 INBOUND region event_in_region
      FILTER other.merged_into == null RETURN 1)
  RETURN {region: region == null ? null : UNSET(region, "_id", "_rev"),
          network: network == null ? null : UNSET(network, "_id", "_rev"),
          same_region: same_region, region_event_count: region_total}
```

Bind variables: {"key": "usgs_aka2026szyrbl", "limit": 10}


```
[
  {
    "region": {
      "_key": "alaska",
      "kind": "us_state",
      "method_version": 1,
      "name": "Alaska"
    },
```

```
    "network": {
      "_key": "ak",
      "code": "ak",
      "name": "Alaska Earthquake Center"
    },
    "same_region": [
      {
        "depth_km": 13.7,
        "magnitude": 1.5,
        "occurred_at": "2026-10-11T16:55:32.687Z",
        "place": "4 km NNE of Susitna North, Alaska",
        "key": "usgs  aka2026ueabjp"
      },
      {
        "depth_km": 5,
        "magnitude": 1.3,
        "occurred_at": "2026-10-11T16:50:17.579Z",
        "place": "22 km NE of Minto, Alaska",
        "key": "usgs_aka2026udzwvf"
      },
      "..."
    ],
    "region_event_count": 983
  }
]
```


Result: 1 row(s) returned in 31.85 ms. **Index chosen by the optimizer:** edge.

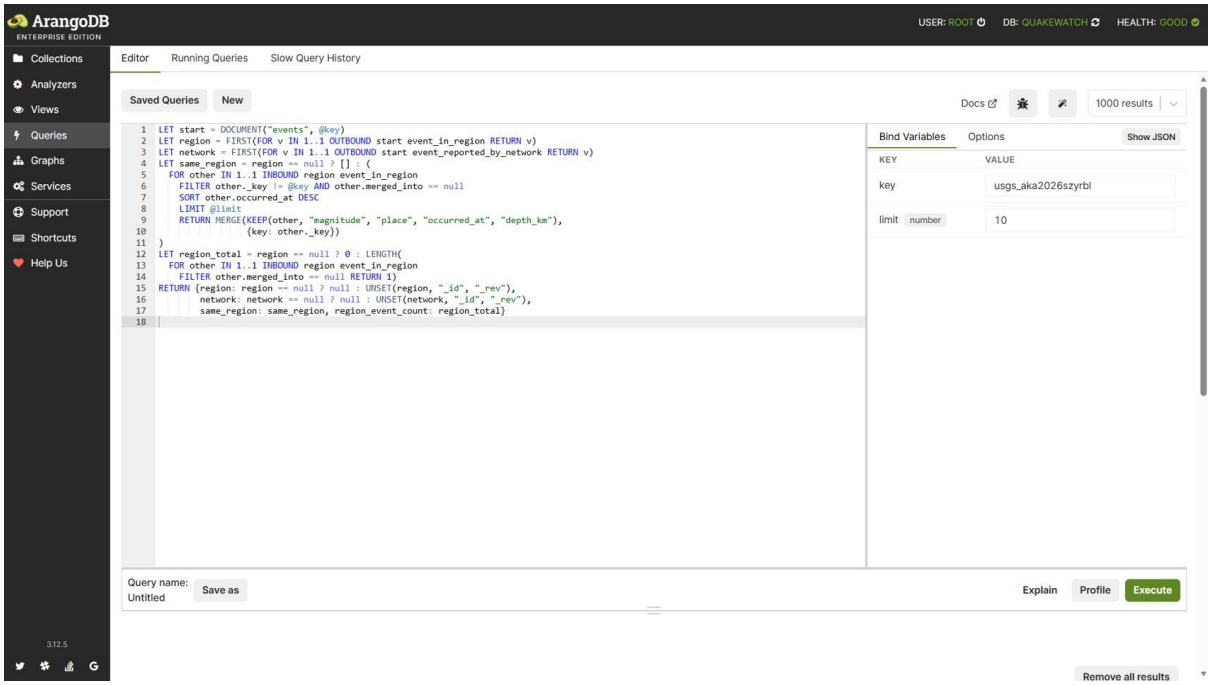


Figure 25: Query 9: Query editor with the relationship AQL (event usgs_aka2026szyrbl).

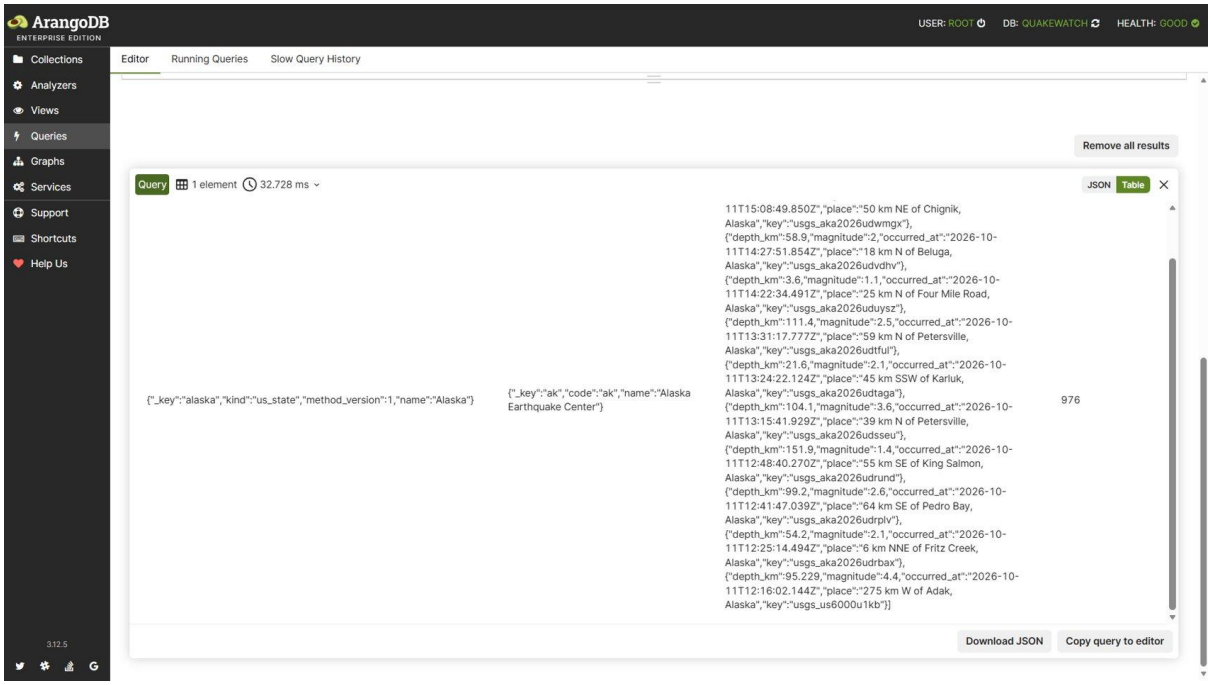


Figure 26: Query 9: Result: region, network and events in the same region.

3.10 Query 10: Map viewport search (GEO_CONTAINS)

Catalogue name: events_in_viewport. **Purpose:** Events inside a map viewport bounding box. **Techniques:** geo index, GEO_CONTAINS.

```
LET total = FIRST(
FOR e IN events
  FILTER GEO_CONTAINS(@bbox0, e.coordinates)
  FILTER e.merged_into == null
  COLLECT WITH COUNT INTO n
  RETURN n
)
LET rows = (
FOR e IN events
  FILTER GEO_CONTAINS(@bbox0, e.coordinates)
  FILTER e.merged_into == null
  SORT e.occurred_at DESC, e._key DESC
  LIMIT @offset, @limit
  RETURN MERGE(KEEP(e, @fields), {key: e._key, region_name: e.derived.region_name})
)
RETURN {total: total, rows: rows}
```

Bind variables: {"bbox0": {"type": "Polygon", "coordinates": [[[-170.0, 50.0], [-130.0, 50.0], [-130.0, 72.0], [-170.0, 72.0], [-170.0, 50.0]]]}, "fields": ["source_event_id", "magnitude", "magnitude_type", "place", "occurred_at", "depth_km", "latitude", "longitude", "alert", "status", "tsunami", "network", "significance", "event_type", "felt", "revision_count", "source_updated_at"], "offset": 0, "limit": 25}

```
[
  {
    "total": 750,
    "rows": [
```

```
    {
      "alert": null,
      "depth_km": 72.7,
      "event_type": "earthquake",
      "felt": null,
      "latitude": 60.87,
      "longitude": -151.479,
      "magnitude": 2.4,
      "magnitude_type": "ml",
      "network": "ak",
      "occurred_at": "2026-10-11T16:45:11.678Z",
      "place": "22 km NNW of Nikiski, Alaska",
      "revision_count": 1,
      "significance": 89,
      "source_event_id": "aka2026udzskb",
      "source_updated_at": "2026-10-11T16:48:40.440Z",
      "status": "automatic",
      "tsunami": false,
      "region_name": "Alaska",
      "key": "usgs_aka2026udzskb"
    },
    {
      "alert": null,
      "depth_km": 73.1,
      "event_type": "earthquake",
      "felt": null,
      "latitude": 62.952,
      "longitude": -149.616,
      "magnitude": 2,
      "magnitude_type": "ml",
      "network": "ak",
      "occurred_at": "2026-10-11T16:36:41.635Z",
      "place": "59 km SW of Cantwell, Alaska",
      "revision_count": 2,
      "significance": 62,
... (truncated for the report)
```


Result: 1 row(s) returned in 9.77 ms. **Index chosen by the optimizer:** qw_events_geo.

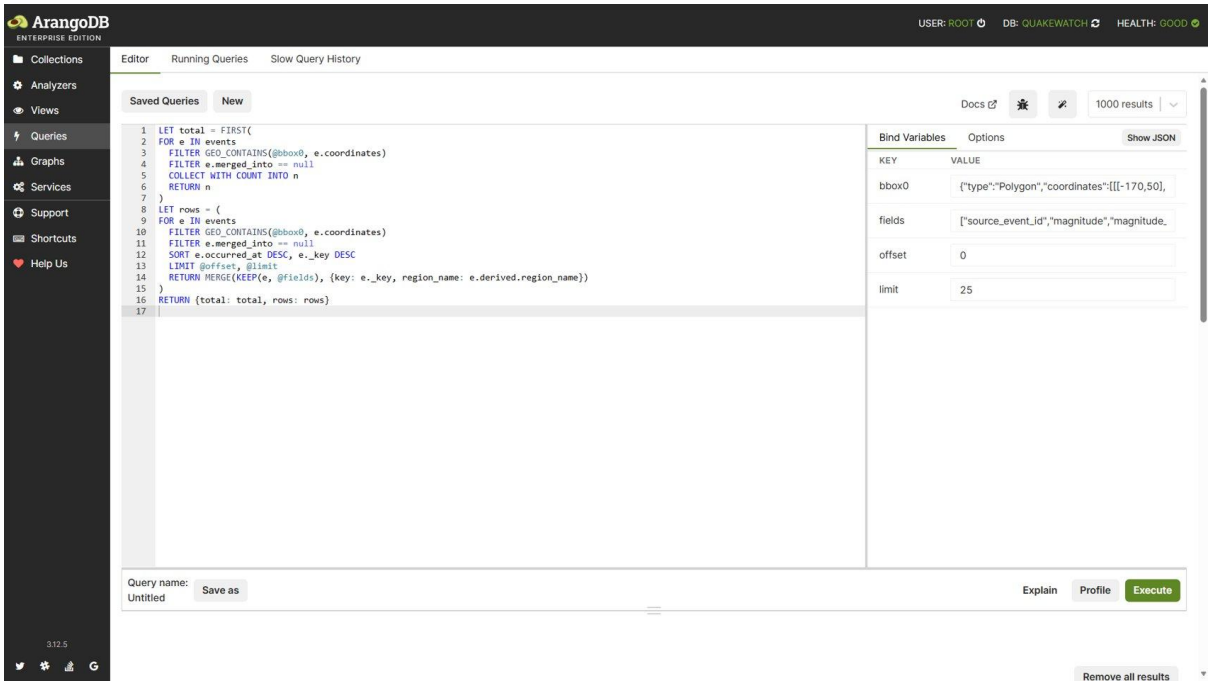


Figure 27: Query 10: Query editor with the GEO_CONTAINS viewport AQL.

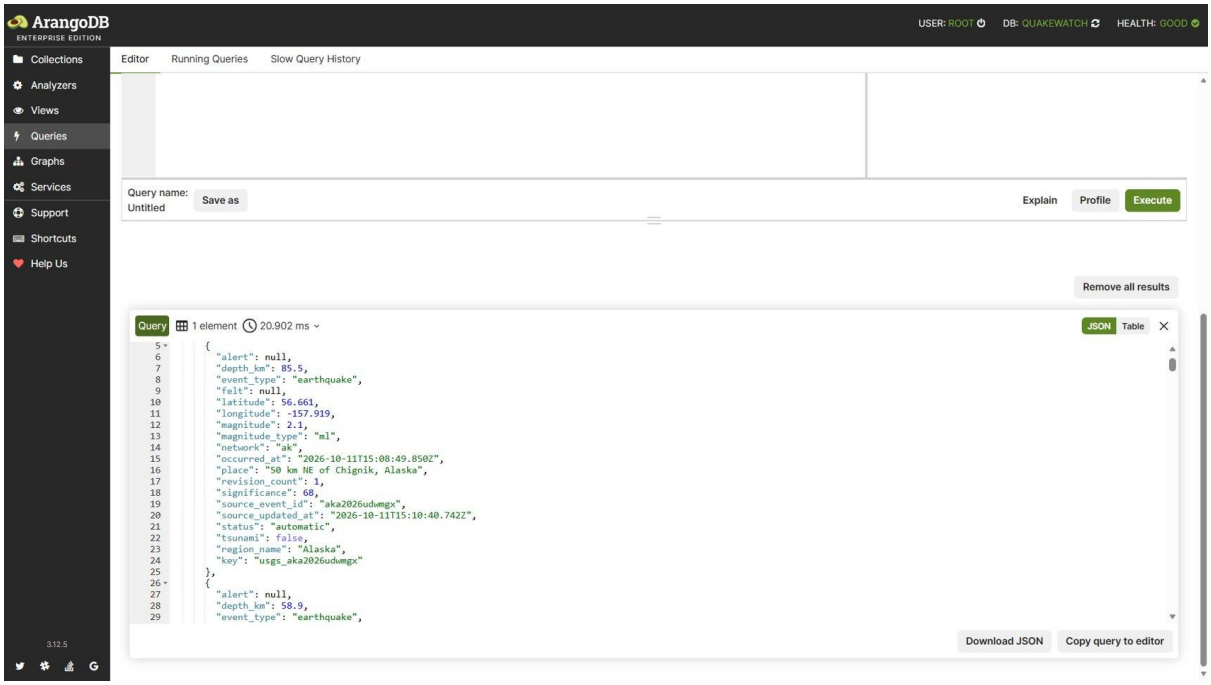


Figure 28: Query 10: Result: total and the first matching events.

3.11 Query 11: Radius search (GEO_DISTANCE)

Catalogue name: events_within_radius. **Purpose:** Events within a geographic radius of a point (README Q2). **Techniques:** geo index, GEO_DISTANCE.

```
LET total = FIRST(
FOR e IN events
  FILTER GEO_DISTANCE(e.coordinates, [@lon, @lat]) <= @radius_m
  FILTER e.magnitude >= @min_mag
  FILTER e.merged_into == null
  COLLECT WITH COUNT INTO n
  RETURN n
)
LET rows = (
FOR e IN events
  FILTER GEO_DISTANCE(e.coordinates, [@lon, @lat]) <= @radius_m
  FILTER e.magnitude >= @min_mag
  FILTER e.merged_into == null
  SORT e.occurred_at DESC, e._key DESC
  LIMIT @offset, @limit
  RETURN MERGE(KEEP(e, @fields), {key: e._key, region_name: e.derived.region_name})
)
RETURN {total: total, rows: rows}
```

Bind variables: {"lon": -149.9, "lat": 61.2, "radius_m": 300000.0, "min_mag": 2.5, "fields": ["source_event_id", "magnitude", "magnitude_type", "place", "occurred_at", "depth_km", "latitude", "longitude", "alert", "status", "tsunami", "network", "significance", "event_type", "felt", "revision_count", "source_updated_at"], "offset": 0, "limit": 25}

```
[
  {
    "total": 93,
    "rows": [
```

```
    {
      "alert": null,
      "depth_km": 111.4,
      "event_type": "earthquake",
      "felt": null,
      "latitude": 63.026,
      "longitude": -150.653,
      "magnitude": 2.5,
      "magnitude_type": "ml",
      "network": "ak",
      "occurred_at": "2026-10-11T13:31:17.777Z",
      "place": "59 km N of Petersville, Alaska",
      "revision_count": 2,
      "significance": 96,
      "source_event_id": "aka2026udtful",
      "source_updated_at": "2026-10-11T13:34:19.703Z",
      "status": "automatic",
      "tsunami": false,
      "region_name": "Alaska",
      "key": "usgs_aka2026udtful"
    },
    {
      "alert": null,
      "depth_km": 104.1,
      "event_type": "earthquake",
      "felt": null,
      "latitude": 62.853,
      "longitude": -150.688,
      "magnitude": 3.6,
      "magnitude_type": "ml",
      "network": "ak",
      "occurred_at": "2026-10-11T13:15:41.929Z",
      "place": "39 km N of Petersville, Alaska",
      "revision_count": 2,
... (truncated for the report)
```


Result: 1 row(s) returned in 3.52 ms. **Index chosen by the optimizer:** qw_events_geo.

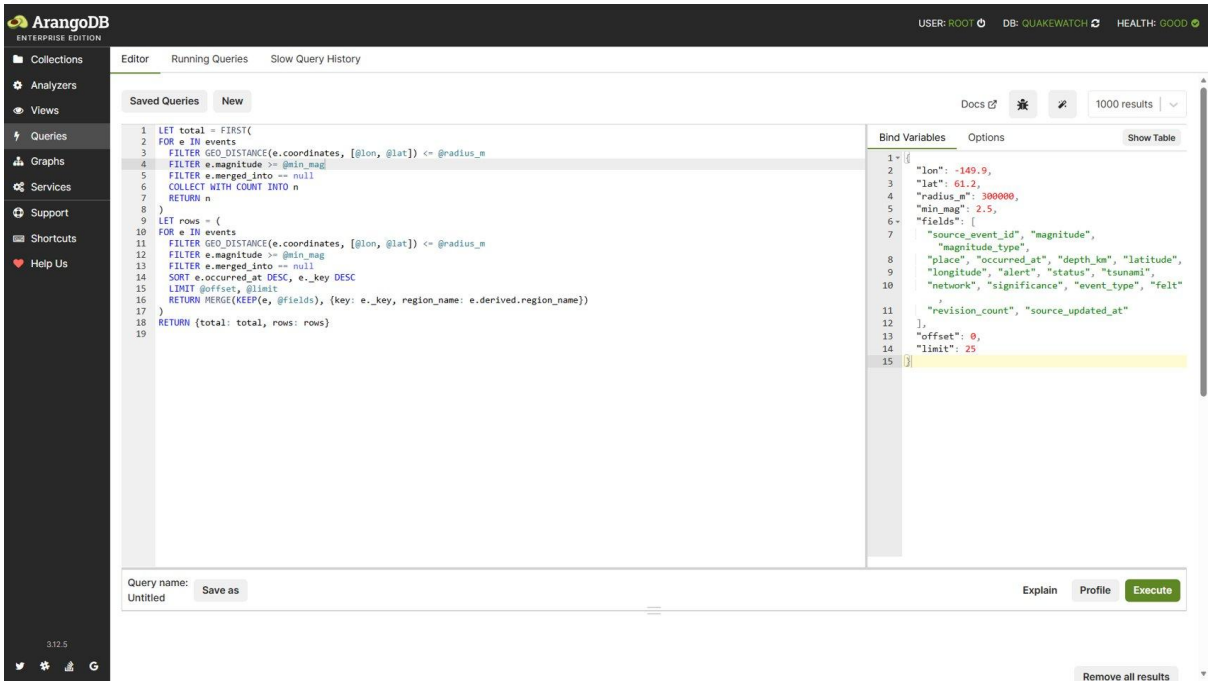


Figure 29: Query 11: Query editor with the GEO_DISTANCE radius AQL.

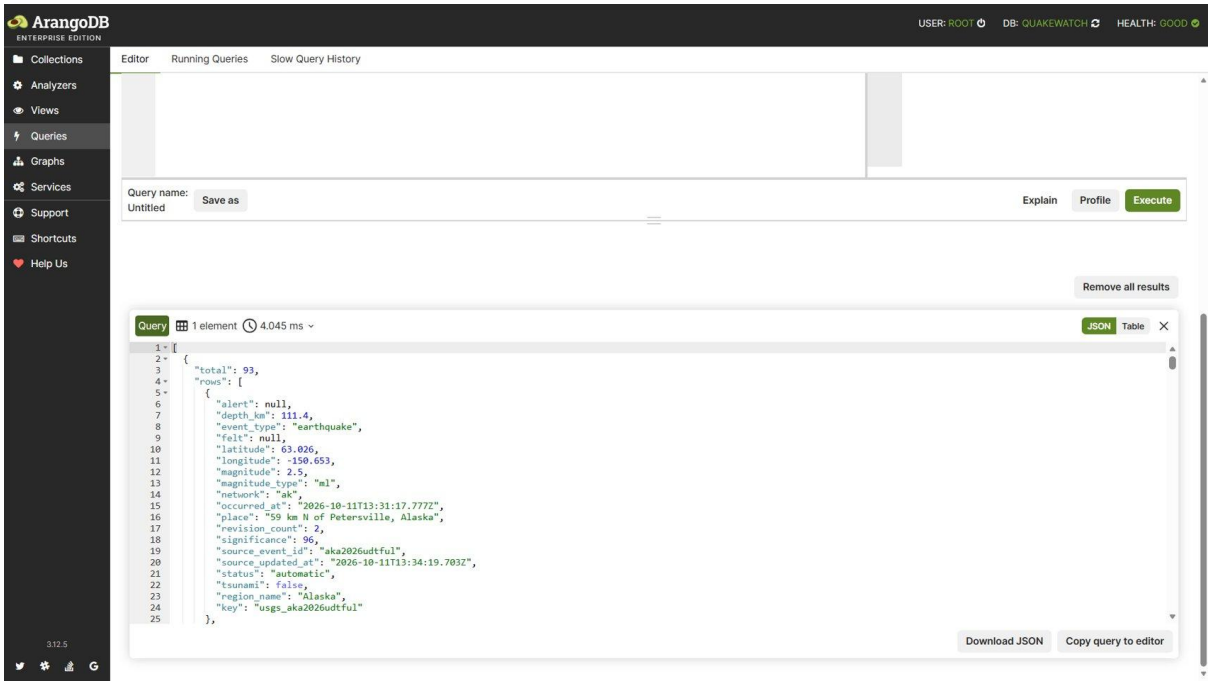


Figure 30: Query 11: Result: 93 events within 300 km of Anchorage with magnitude 2.5 or more.

3.12 Query 12: Location text search (ArangoSearch view)

Catalogue name: events_location_text. **Purpose:** Full-text location search over place and derived region (ArangoSearch view). **Techniques:** ArangoSearch view, text analyzer.

```
LET total = FIRST(
FOR e IN events_search SEARCH ANALYZER(TOKENS(@q, 'text_en') ALL == e.place, 'text_en') OR
ANALYZER(TOKENS(@q, 'text_en') ALL == e.derived.region_name, 'text_en')
  FILTER e.merged_into == null
  COLLECT WITH COUNT INTO n
  RETURN n
)
LET rows = (
FOR e IN events_search SEARCH ANALYZER(TOKENS(@q, 'text_en') ALL == e.place, 'text_en') OR
ANALYZER(TOKENS(@q, 'text_en') ALL == e.derived.region_name, 'text_en')
  FILTER e.merged_into == null
  SORT e.occurred_at DESC, e._key DESC
  LIMIT @offset, @limit
  RETURN MERGE(KEEP(e, @fields), {key: e._key, region_name: e.derived.region_name})
)
RETURN {total: total, rows: rows}
```

Bind variables: {"q": "fiji islands", "fields": ["source_event_id", "magnitude", "magnitude_type", "place", "occurred_at", "depth_km", "latitude", "longitude", "alert", "status", "tsunami", "network", "significance", "event_type", "felt", "revision_count", "source_updated_at"], "offset": 0, "limit": 25}


```
[
  {
    "total": 20,
    "rows": [
      {
        "alert": null,
        "depth_km": 546.339,
```

```
    "event_type": "earthquake",
    "felt": null,
    "latitude": -23.6919,
    "longitude": 179.9648,
    "magnitude": 4.5,
    "magnitude_type": "mb",
    "network": "us",
    "occurred_at": "2026-10-11T07:28:52.878Z",
    "place": "south of the Fiji Islands",
    "revision_count": 1,
    "significance": 312,
    "source_event_id": "us6000u1j3",
    "source_updated_at": "2026-10-11T07:46:37.040Z",
    "status": "reviewed",
    "tsunami": false,
    "region_name": "south of the Fiji Islands",
    "key": "usgs_us6000u1j3"
  },
  {
    "alert": null,
    "depth_km": 587.386,
    "event_type": "earthquake",
    "felt": null,
    "latitude": -22.071,
    "longitude": -179.5886,
    "magnitude": 4.6,
    "magnitude_type": "mb",
    "network": "us",
    "occurred_at": "2026-10-10T13:40:44.101Z",
    "place": "south of the Fiji Islands",
    "revision_count": 1,
... (truncated for the report)
```


Result: 1 row(s) returned in 4.11 ms. **Index chosen by the optimizer:** ArangoSearch view (no secondary index).

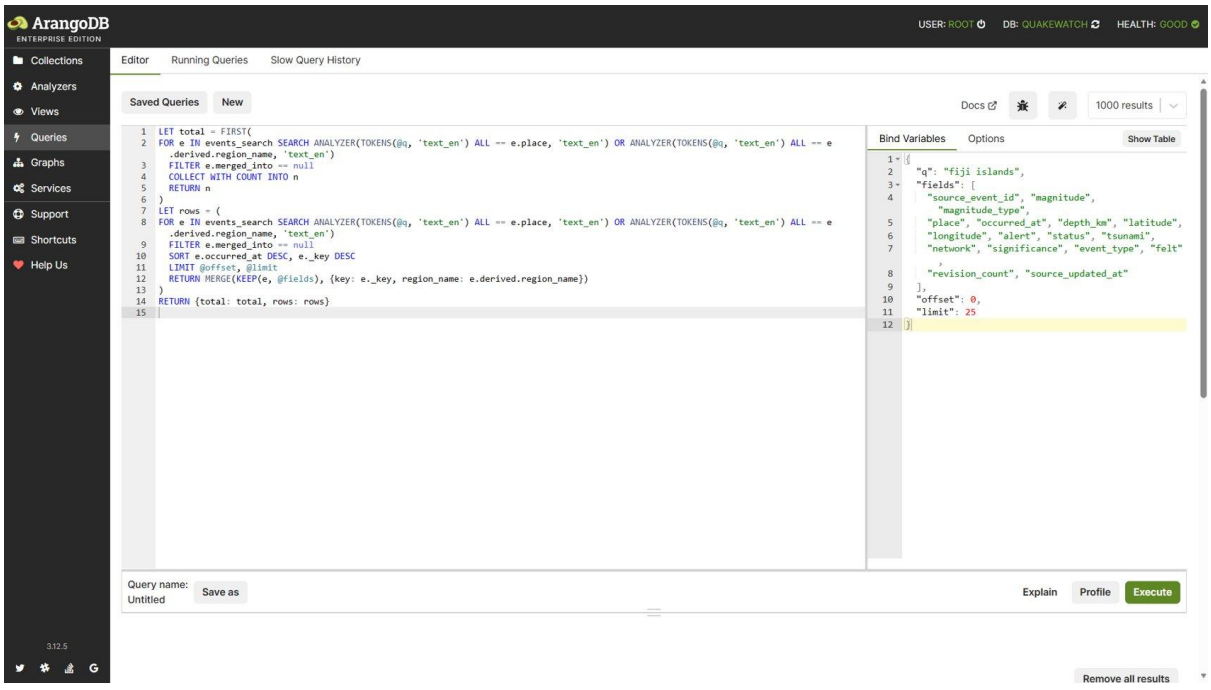


Figure 31: Query 12: Query editor with the ArangoSearch AQL (q = fiji islands).

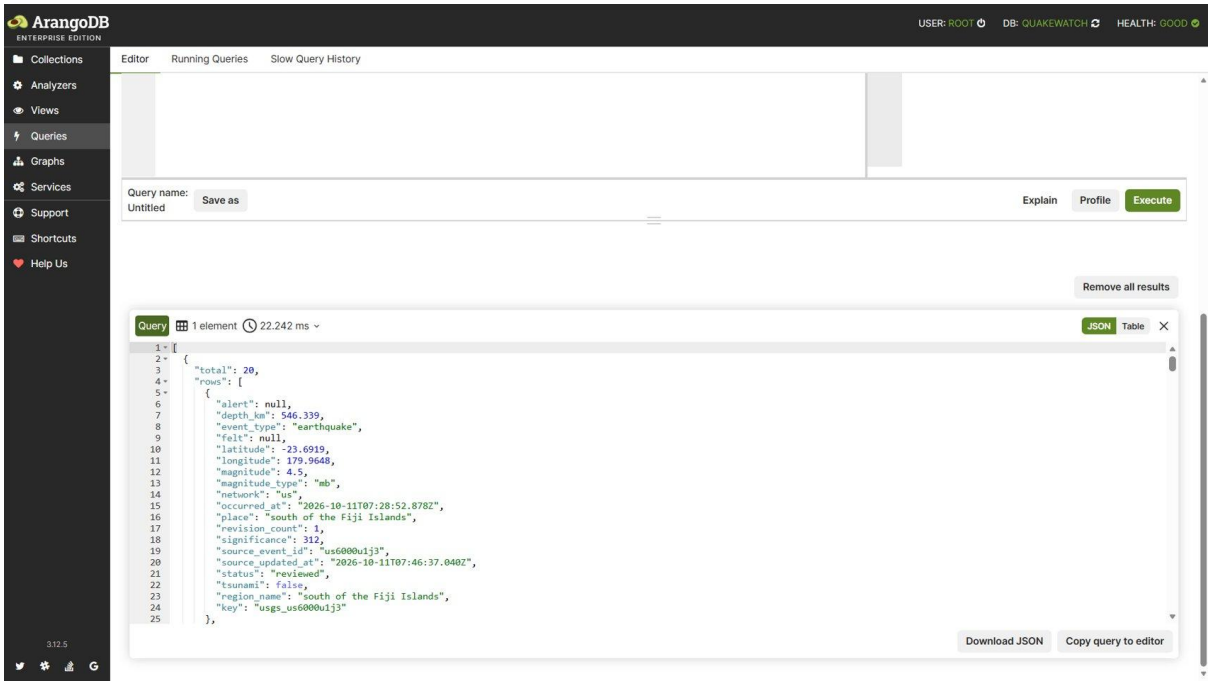


Figure 32: Query 12: Result: 20 matching events.

3.13 Query 13: Ingestion run statistics by kind

Catalogue name: ops_run_stats. **Purpose:** Run outcomes and durations per kind since a timestamp (success rate, p95). **Techniques:** aggregation, percentiles, array operators.

```
FOR r IN ingestion_runs
  FILTER r.started_at >= @since
  COLLECT kind = r.kind INTO runs = r
  LET durations = runs[* FILTER CURRENT.duration_ms != null RETURN CURRENT.duration_ms]
  RETURN {
    kind: kind,
    total: LENGTH(runs),
    by_status: MERGE(FOR st IN UNIQUE(runs[*].status)
                  RETURN {[st]: LENGTH(runs[* FILTER CURRENT.status == st])}),
    p50_ms: LENGTH(durations) == 0 ? null : PERCENTILE(durations, 50),
    p95_ms: LENGTH(durations) == 0 ? null : PERCENTILE(durations, 95)
  }
```

Bind variables: {"since": "2026-09-01T00:00:00.000Z"}


```
[
  {
    "kind": "backfill_window",
    "total": 6,
    "by_status": {
      "failed": 1,
      "succeeded": 5
    },
    "p50_ms": 14509.7,
    "p95_ms": 25695.6
  },
  {
    "kind": "detail_enrichment",
    "total": 8,
    "by_status": {
```

```
    "succeeded": 8
   },
   "p50_ms": 561.9,
   "p95_ms": 1335.3
  },
  {
   "kind": "feed  sync",
   "total": 135,
   "by_status": {
    "succeeded": 135
   },
   "p50  ms": 720.5,
   "p95_ms": 1660.6
  }
]
```


Result: 3 row(s) returned in 2.04 ms. **Index chosen by the optimizer:** qw_runs_kind_started.

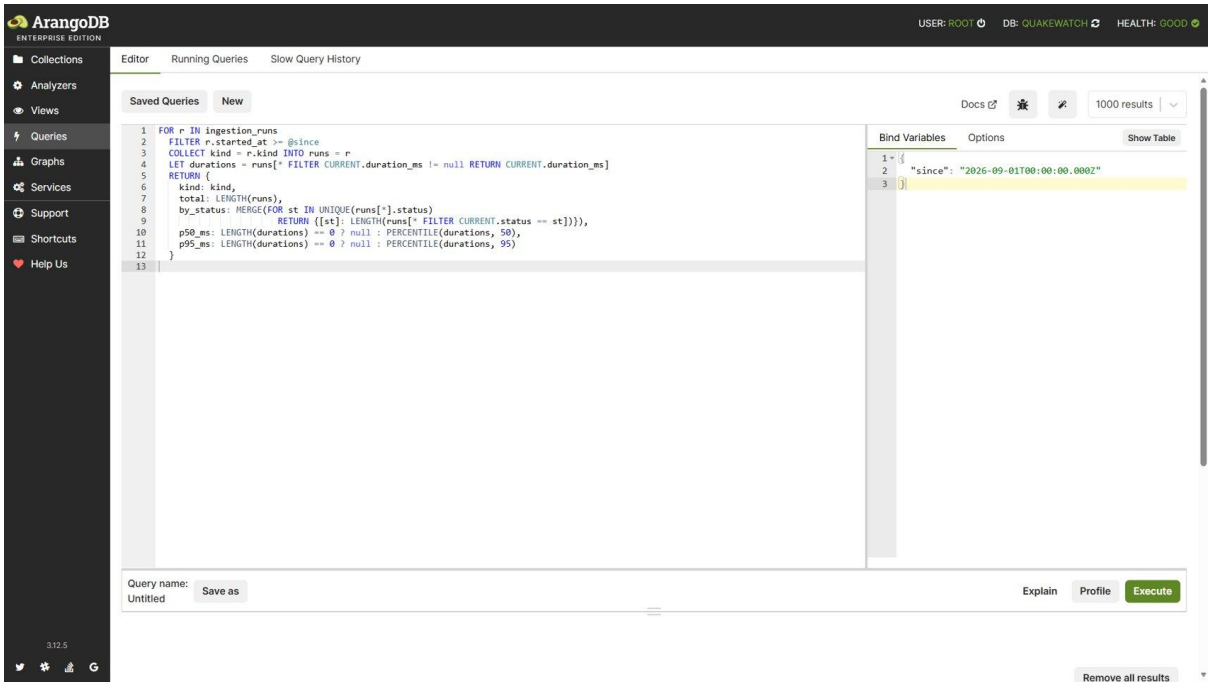


Figure 33: Query 13: Query editor with the run-statistics AQL.

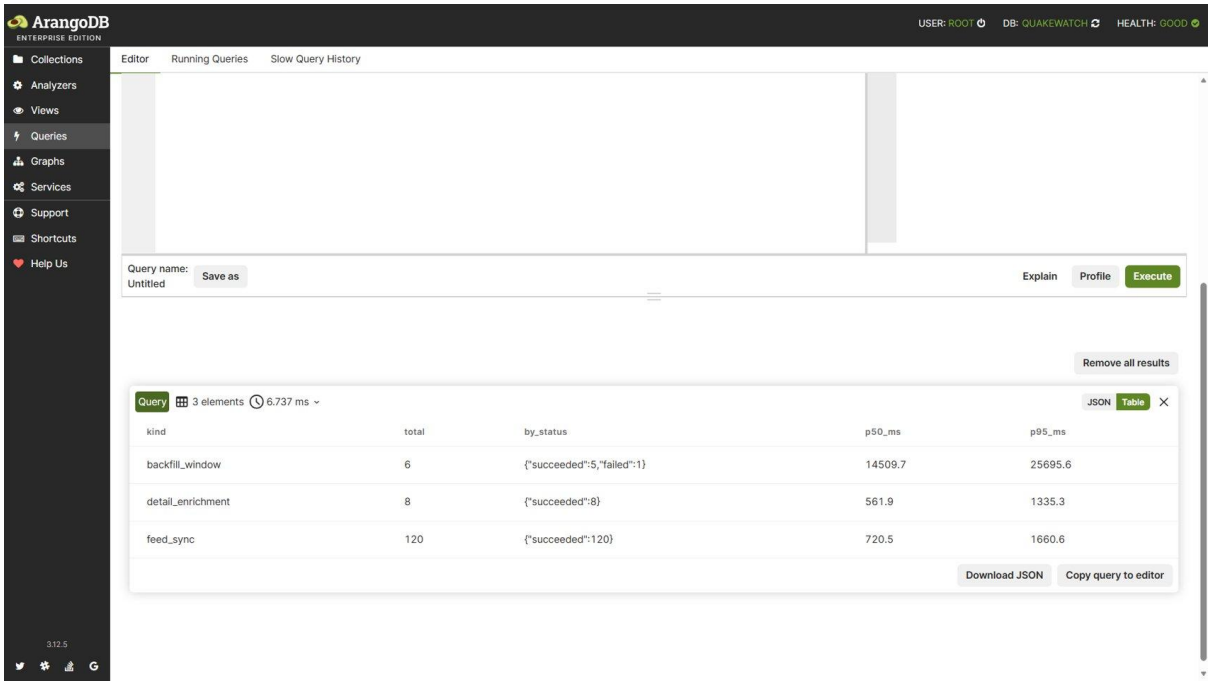


Figure 34: Query 13: Result: per-kind run counts, status mix and latency percentiles.

3.14 Query 14: Depth histogram (bucketed aggregation)

Catalogue name: analytics_depth_histogram. **Purpose:** Depth distribution (shallow < 70 km, intermediate 70 to 300 km, deep > 300 km, 10 km bins). **Techniques:** aggregation.

```
FOR e IN events
  FILTER e.occurred_at >= @since FILTER e.merged_into == null FILTER e.depth_km != null
  COLLECT bin = FLOOR(e.depth_km / 10) * 10 WITH COUNT INTO n
  SORT bin
  RETURN {bin: bin, count: n}
```

Bind variables: {"since": "2026-08-26T00:00:00.000Z"}


```
[
  {
    "bin": -10,
    "count": 27
  },
  {
    "bin": 0,
    "count": 683
  },
  {
    "bin": 10,
    "count": 910
  }
]
```


Result: 59 row(s) returned in 8.96 ms. **Index chosen by the optimizer:** qw_events_occurred_at.

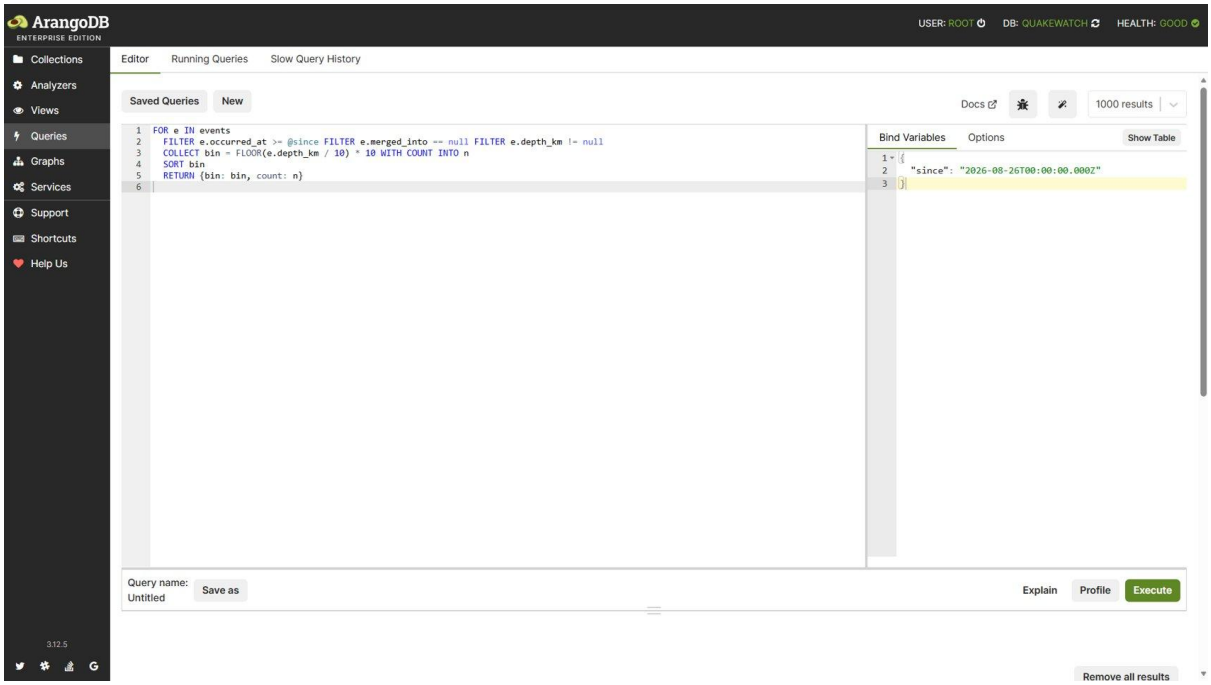


Figure 35: Query 14: Query editor with the depth histogram AQL.

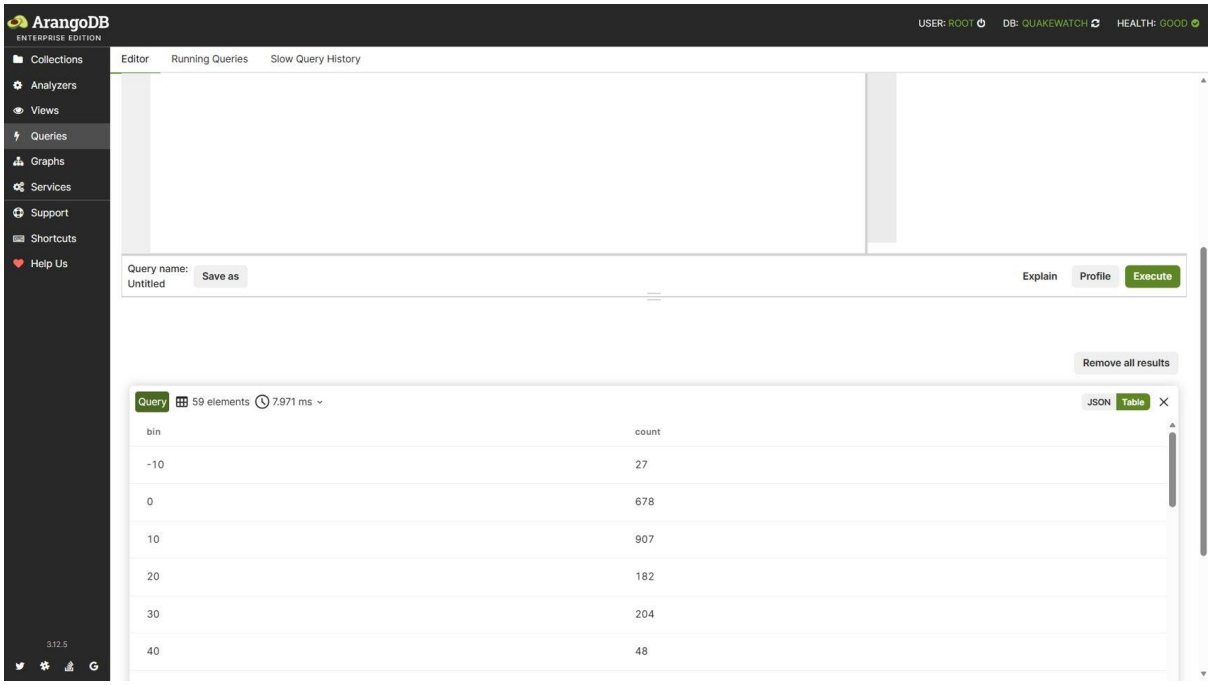


Figure 36: Query 14: Result: 59 depth bins of 10 km.

3.15 Query 15: Nearby events in space and time

Catalogue name: event_nearby_space_time. **Purpose:** Other events within a radius and a time window of an event (proximity only; not an aftershock classification — ESD §20C). **Techniques:** geo index, GEO_DISTANCE, time window.

```
FOR e IN events
  FILTER GEO_DISTANCE(e.coordinates, @point) <= @radius_m
```

```
    FILTER e.occurred_at >= @start AND e.occurred_at <= @end
    FILTER e._key != @key AND e.merged_into == null
    LET distance_km = GEO_DISTANCE(e.coordinates, @point) / 1000
    SORT distance_km ASC
    LIMIT @limit
    RETURN {key: e._key, magnitude: e.magnitude, place: e.place,
          occurred_at: e.occurred_at, distance_km: ROUND(distance_km * 10) / 10}
```

Bind variables: {"point": [-150.0, 61.0], "radius_m": 100000, "start": "2026-09-01T00:00:00.000Z", "end": "2026-09-15T00:00:00.000Z", "key": "usgs_x", "limit": 10}


```
[
  {
    "key": "usgs_aka2026rqamip",
    "magnitude": 2.9,
    "place": "3 km N of Anchorage, Alaska",
    "occurred_at": "2026-09-05T23:31:38.296Z",
    "distance_km": 28.4
  },
  {
    "key": "usgs_aka2026sbenku",
    "magnitude": 2.7,
    "place": "21 km SSW of Sunrise, Alaska",
    "occurred_at": "2026-09-12T00:28:57.062Z",
    "distance_km": 37.1
  },
  {
    "key": "usgs_aka2026rxarpv",
    "magnitude": 2.9,
    "place": "25 km ENE of Beluga, Alaska",
    "occurred_at": "2026-09-09T18:35:05.415Z",
    "distance_km": 44.8
  }
]
```


Result: 7 row(s) returned in 1.56 ms. **Index chosen by the optimizer:** qw_events_geo.

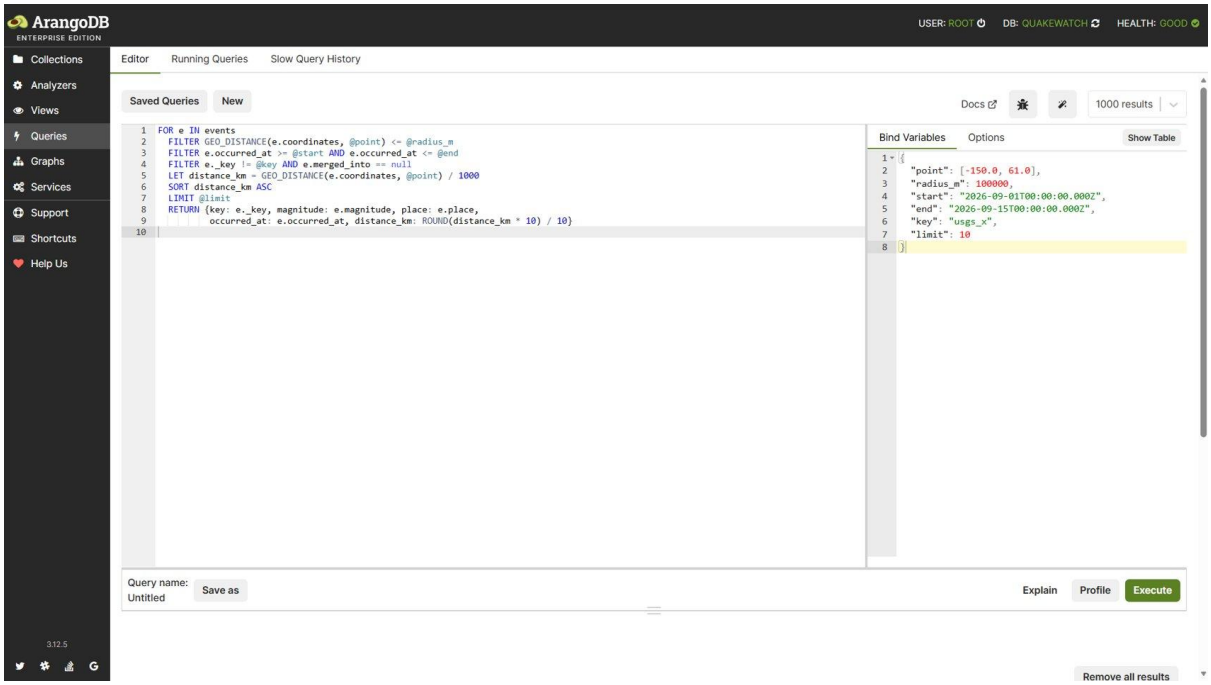


Figure 37: Query 15: Query editor with the space-time AQL.

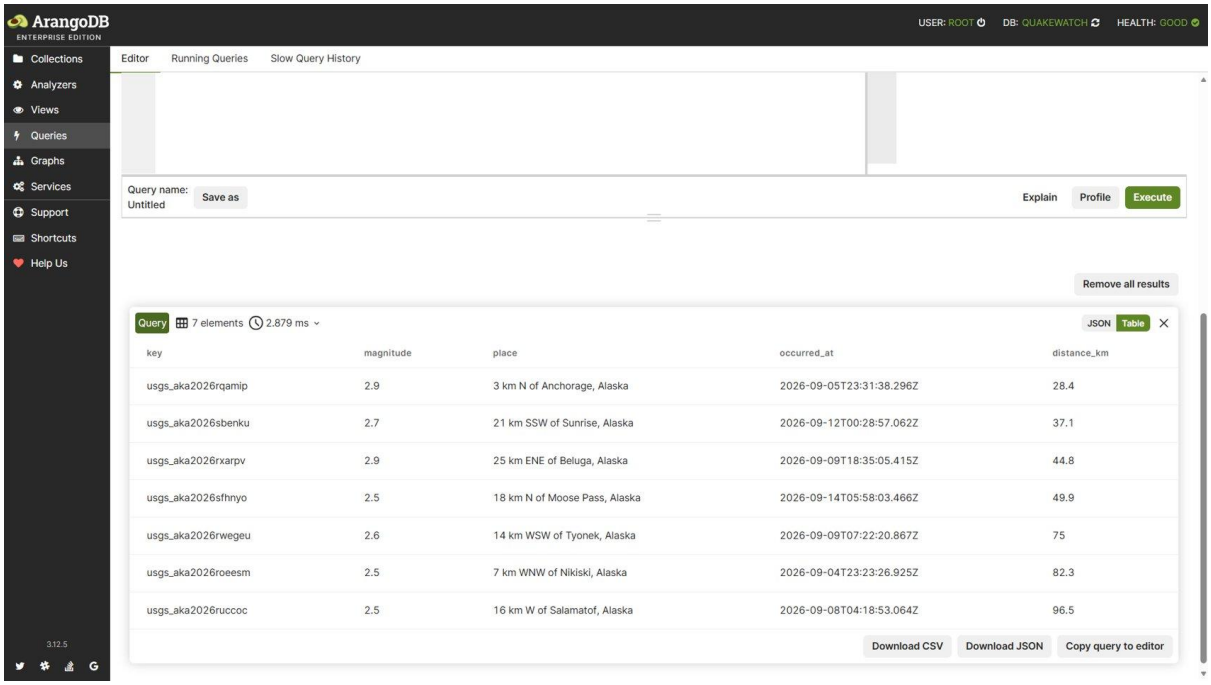


Figure 38: Query 15: Result: seven events within 100 km and the time window of the reference point.

3.16 Summary of the query execution

Table 8: Fifteen AQL queries executed against the live database

#	Query	Rows	Time (ms)	Index used
1	analytics_avg_magnitude_by_region	20	6.98	qw_events_occurred_at
2	analytics_dashboard_summary	1	32.24	qw_events_occurred_at
3	analytics_network_stats	15	6.83	qw_events_occurred_at
4	analytics_magnitude_changes	10	3.15	qw_revisions_changed_fields

#	Query	Rows	Time (ms)	Index used
5	analytics_ingestion_latency	1	3.29	qw_events_occurred_at, qw_runs_kind_started
6	analytics_significant_by_week	6	9.78	qw_events_occurred_at
7	alerts_user_rules	1	2.18	edge, qw_rules_user
8	alerts_matched_events_graph	1	1.22	edge
9	event_graph_relationships	1	31.85	edge
10	events_in_viewport	1	9.77	qw_events_geo
11	events_within_radius	1	3.52	qw_events_geo
12	events_location_text	1	4.11	ArangoSearch view (no secondary index)
13	ops_run_stats	3	2.04	qw_runs_kind_started
14	analytics_depth_histogram	59	8.96	qw_events_occurred_at
15	event_nearby_space_time	7	1.56	qw_events_geo

4. Backend and Database Connectivity: CRUD Demonstration

4.1 Authentication and session mechanism

The code base was inspected before any request was written. The API uses Django REST Framework SessionAuthentication; there is no JWT and no bearer token. The mechanism is therefore:

- **csrftoken cookie.** Any page request (here GET /accounts/login/) sets the csrftoken cookie.
- **Login.** POST /accounts/login/ with the form fields username, password and csrfmiddlewaretoken. On success the server answers 302 to / and sets an HttpOnly sessionid cookie; the session document is stored in the ArangoDB sessions collection (a TTL index expires it after seven days). Django rotates the csrftoken at this moment.
- **Unsafe methods.** Every POST, PATCH and DELETE to /api/v1/ must carry the header X-CSRFToken equal to the current csrftoken cookie; otherwise DRF answers 403 'CSRF Failed'.
- **Authorisation.** Alert rules, saved searches and snapshots are owner-scoped: a user can only read or change documents carrying their own user_id. Operations endpoints additionally require staff status.
- **Throttling.** Request counters are kept in ArangoDB (600/min per user, snapshot creation 10/hour).

Cookies and tokens are handled by the Postman cookie jar and by scripts in the collection; their values are never printed in this report (shown as <hidden>).

4.2 Backend and database connectivity check

Request 03 calls the public health endpoint, which pings ArangoDB and the cache and reads the freshness of the last successful USGS synchronisation. A direct health probe of the database layer returned
{"status":"ok","checks":{"arangodb":{"ok":true,"version":"3.12.5"},"schema":{"ok":true,"applied":6,"expected":6}}}.

4.3 Request sequence and verified results

Fifteen requests were executed in order against http://localhost:8000. After each write and each delete, the target document was read back from ArangoDB directly (HTTP cursor API with the database credentials, which are not shown) to prove that the API call changed the

stored state. The requests span five resources: sessions/accounts, events, saved searches, alert rules and research snapshots. A disposable account with the same privileges as the demonstration account was used for the evidence run and removed afterwards.

Table 9: Fifteen HTTP requests and their verified status codes

#	Method	Path	Operation	Expected	Actual	Result
01	GET	/accounts/login/	AUTH	200	200	Pass
02	POST	/accounts/login/	AUTH / CREATE session	302	302	Pass
03	GET	/api/v1/system/health	READ	200	200	Pass
04	GET	/api/v1/events	READ (filter / sort / paginate)	200	200	Pass
05	POST	/api/v1/saved-searches	CREATE	201	201	Pass
06	GET	/api/v1/saved-searches/265750	READ	200	200	Pass
07	PATCH	/api/v1/saved-searches/265750	UPDATE	200	200	Pass
08	POST	/api/v1/alert-rules	CREATE	201	201	Pass
09	PATCH	/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e	UPDATE	200	200	Pass
10	GET	/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e/preview	READ	200	200	Pass
11	POST	/api/v1/research/snapshots	CREATE	201	201	Pass
12	DELETE	/api/v1/research/snapshots/e748993f062e4ae6b38171f43fba1e6b	DELETE	204	204	Pass
13	DELETE	/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e	DELETE	204	204	Pass
14	DELETE	/api/v1/saved-searches/265750	DELETE	204	204	Pass
15	GET	/api/v1/saved-searches/265750	READ (negative)	404	404	Pass

Outcome: 15 of 15 requests passed. The same sequence was then executed with the Newman runner of Postman (15 requests, 35 assertions, 0 failures).

4.3.1 Request 01: Open login page (obtain CSRF cookie)

```
GET http://localhost:8000/accounts/login/
Accept: application/json
```

```
HTTP 200 (expected 200), server time 2.18 ms
HTML login form (200 OK); the csrftoken cookie is set.
```

Database operation: Session middleware issues the csrftoken cookie; no DB write.

4.3.2 Request 02: Log in (create session)

```
POST http://localhost:8000/accounts/login/
Accept: application/json
Content-Type: application/x-www-form-urlencoded
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (form): username=<demo account>&password=********&csrfmiddlewaretoken=<hidden>
```

```
HTTP 302 (expected 302), server time 288.79 ms
Empty body; Location: /; sessionid cookie set (HttpOnly).
```

Database operation: Credentials checked against `users`; session document written to `sessions`.

Direct read-back from ArangoDB (auth_users + sessions after login): {"user_exists": true, "live_sessions": 6}

4.3.3 Request 03: System health (backend + DB connectivity)

```
GET http://localhost:8000/api/v1/system/health
Accept: application/json
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 200 (expected 200), server time 14.83 ms
{
  "status": "ok",
  "database": "up",
  "cache": "up",
  "freshness": "fresh",
  "checked_at": "2026-10-11T16:48:27.241Z"
}
```

Database operation: Pings ArangoDB and cache; reads freshness from `ingestion_runs`.

4.3.4 Request 04: List and filter earthquake events

```
GET http://localhost:8000/api/v1/events?min_magnitude=4.5&sort=magnitude&direction=desc&page_size=3
Accept: application/json
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 200 (expected 200), server time 9.07 ms
{
  "results": [
    {
      "key": "usgs_us6000txpi",
      "magnitude": 6.6,
      "place": "80 km ENE of Tadine, New Caledonia",
      "occurred_at": "2026-09-25T21:23:03.309Z"
    },
    {
      "key": "usgs_us7000ti1p",
      "magnitude": 6.5,
      "place": "177 km W of Nikolski, Alaska",
      "occurred_at": "2026-09-17T14:19:52.472Z"
    },
    "..."
  ],
  "page": 1,
  "page_size": 3,
  "total": 553,
  "total_pages": 185,
  "query": {
    "filters": {
      "direction": "desc",
      "min_magnitude": 4.5,
      "sort": "magnitude"
    },
    "summary": "magnitude 4.5 to any — sorted by magnitude desc"
  }
}
```

Database operation: AQL over `events` with bind variables; total = full match count.

4.3.5 Request 05: Create saved search

```
POST http://localhost:8000/api/v1/saved-searches
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (JSON):
{
  "name": "Strong quakes M4.5+ (demo)",
  "filters": {
    "min_magnitude": 4.5,
    "sort": "magnitude",
    "direction": "desc"
  }
}
```

```
HTTP 201 (expected 201), server time 16.78 ms
{
  "created_at": "2026-10-11T16:48:31.398Z",
  "filters": {
    "direction": "desc",
    "min_magnitude": 4.5,
    "sort": "magnitude"
  },
  "name": "Strong quakes M4.5+ (demo)",
  "summary": "magnitude 4.5 to any — sorted by magnitude desc",
  "updated_at": "2026-10-11T16:48:31.398Z",
  "user  id": 2,
  "id": "265750"
}
```

Database operation: INSERT into `saved_searches` (per-user cap enforced atomically).

Direct read-back from ArangoDB (saved_searches document after POST): {"filters": {"direction": "desc", "min_magnitude": 4.5, "sort": "magnitude"}, "name": "Strong quakes M4.5+ (demo)", "user_id": 2}

Captured for later requests: saved_search_id = 265750

4.3.6 Request 06: Read saved search (persistence check)

```
GET http://localhost:8000/api/v1/saved-searches/265750
Accept: application/json
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 200 (expected 200), server time 8.92 ms
{
  "created_at": "2026-10-11T16:48:31.398Z",
  "filters": {
    "direction": "desc",
    "min_magnitude": 4.5,
    "sort": "magnitude"
  },
  "name": "Strong quakes M4.5+ (demo)",
  "summary": "magnitude 4.5 to any — sorted by magnitude desc",
  "updated_at": "2026-10-11T16:48:31.398Z",
  "user_id": 2,
  "id": "265750"
}
```

Database operation: Document fetched from `saved_searches` by key and owner.

4.3.7 Request 07: Rename saved search

```
PATCH http://localhost:8000/api/v1/saved-searches/265750
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (JSON):
{
  "name": "Strong quakes M4.5+ (renamed)"
}
```

```
HTTP 200 (expected 200), server time 11.06 ms
{
  "created_at": "2026-10-11T16:48:31.398Z",
  "filters": {
    "direction": "desc",
    "min_magnitude": 4.5,
    "sort": "magnitude"
  },
  "name": "Strong quakes M4.5+ (renamed)",
  "summary": "magnitude 4.5 to any — sorted by magnitude desc",
  "updated_at": "2026-10-11T16:48:35.524Z",
  "user_id": 2,
  "id": "265750"
}
```

Database operation: UPDATE of `name` and `updated_at` in `saved_searches`.

Direct read-back from ArangoDB (saved_searches document after PATCH): {"name": "Strong quakes M4.5+ (renamed)", "updated_at": "2026-10-11T16:48:35.524Z"}

4.3.8 Request 08: Create alert rule

```
POST http://localhost:8000/api/v1/alert-rules
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (JSON):
{
  "name": "Alaska M2.5+ watch (demo)",
  "enabled": true,
  "cooldown_minutes": 30,
  "conditions": {
    "minimum_magnitude": 2.5,
    "center": [
      -149.9,
      61.2
    ],
    "radius_km": 500
  }
}
```

```
HTTP 201 (expected 201), server time 9.99 ms
{
  "id": "bdcd462b4b9443768aed4a8f0fea109e",
  "user_id": 2,
```

```
  "name": "Alaska M2.5+ watch (demo)",
  "enabled": true,
  "conditions": {
    "minimum_magnitude": 2.5,
    "maximum_depth_km": null,
    "minimum_depth_km": null,
    "center": [
      -149.9,
      61.2
    ],
    "radius_km": 500,
    "alert  levels": [],
    "tsunami_only": false,
    "event_types": [
      "earthquake"
    ]
  },
  "cooldown_minutes": 30,
  "created_at": "2026-10-11T16:48:37.611Z",
  "updated_at": "2026-10-11T16:48:37.611Z"
}
```


Database operation: INSERT into `alert_rules` plus `user_owns_rule` graph edge.

Direct read-back from ArangoDB (alert_rules document and owner edge after POST): {"rule": {"cooldown_minutes": 30, "enabled": true, "name": "Alaska M2.5+ watch (demo)"}, "owner_edges": 1}

Captured for later requests: rule_id = bdcd462b4b9443768aed4a8f0fea109e

4.3.9 Request 09: Update alert rule

```
PATCH http://localhost:8000/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (JSON):
{
  "name": "Alaska M3.0+ watch (updated)",
  "cooldown_minutes": 15,
  "conditions": {
    "minimum_magnitude": 3.0,
    "center": [
      -149.9,
      61.2
    ],
    "radius_km": 500
  }
}
```


```
HTTP 200 (expected 200), server time 23.15 ms
{
  "id": "bdcd462b4b9443768aed4a8f0fea109e",
  "user_id": 2,
  "name": "Alaska M3.0+ watch (updated)",
  "enabled": true,
  "conditions": {
    "minimum_magnitude": 3,
    "maximum_depth_km": null,
    "minimum_depth_km": null,
    "center": [
      -149.9,
```

```
      61.2
    ],
    "radius_km": 500,
    "alert_levels": [],
    "tsunami_only": false,
    "event_types": [
      "earthquake"
    ]
  },
  "cooldown_minutes": 15,
  "created_at": "2026-10-11T16:48:37.611Z",
  "updated  at": "2026-10-11T16:48:39.697Z"
}
```


Database operation: UPDATE of the rule document; validated against the pydantic schema first.

Direct read-back from ArangoDB (alert_rules document after PATCH): {"conditions": {"minimum_magnitude": 3, "maximum_depth_km": null, "minimum_depth_km": null, "center": [-149.9, 61.2], "radius_km": 500, "alert_levels": [], "tsunami_only": false, "event_types": ["earthquake"]}, "cooldown_minutes": 15, "name": "Alaska M3.0+ watch (updated)"}

4.3.10 Request 10: Preview alert rule against stored events

```
GET http://localhost:8000/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e/preview
Accept: application/json
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```


```
HTTP 200 (expected 200), server time 20.36 ms
{
  "rule": {
    "id": "bdcd462b4b9443768aed4a8f0fea109e",
    "name": "Alaska M3.0+ watch (updated)",
    "enabled": true
  },
  "window_days": 7,
  "would_match": "4 event(s) (first omitted)"
}
```


Database operation: Geo/magnitude AQL over `events` using the stored rule conditions.

4.3.11 Request 11: Create research snapshot

```
POST http://localhost:8000/api/v1/research/snapshots
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
Body (JSON):
{
  "title": "M4.5+ research snapshot (demo)",
  "notes": "Frozen selection created from the CRUD demonstration.",
  "filters": {
    "min_magnitude": 4.5,
    "sort": "magnitude",
    "direction": "desc"
  }
}
```

```
HTTP 201 (expected 201), server time 38.65 ms
{
  "id": "e748993f062e4ae6b38171f43fba1e6b",
  "title": "M4.5+ research snapshot (demo)",
  "created_at": "2026-10-11T16:48:43.897Z",
  "row_count": 553,
  "content  sha256": "bc516d5364ab0ecab59c975abd0df84b6610ac7637a7c97ee3356530b18de091",
  "query_definition": {
    "filters": {
      "direction": "desc",
      "min_magnitude": 4.5,
      "sort": "magnitude"
    },
    "summary": "magnitude 4.5 to any — sorted by magnitude desc"
  },
  "application_version": "dev"
}
```

Database operation: Freezes matching event revisions into `research_snapshots` with a SHA-256 checksum.

Direct read-back from ArangoDB (research_snapshots document after POST): {"title": "M4.5+ research snapshot (demo)", "members": 553, "content_sha256": "bc516d5364ab0ecab59c975abd0df84b6610ac7637a7c97ee3356530b18de091"}

Captured for later requests: snapshot_id = e748993f062e4ae6b38171f43fba1e6b

4.3.12 Request 12: Delete research snapshot

```
DELETE http://localhost:8000/api/v1/research/snapshots/e748993f062e4ae6b38171f43fba1e6b
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 204 (expected 204), server time 11.98 ms
Empty body (no content).
```

Database operation: REMOVE from `research_snapshots`.

Direct read-back from ArangoDB (research_snapshots lookup after DELETE): null

4.3.13 Request 13: Delete alert rule

```
DELETE http://localhost:8000/api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 204 (expected 204), server time 13.96 ms
Empty body (no content).
```

Database operation: REMOVE rule and its `user_owns_rule` edge.

Direct read-back from ArangoDB (alert_rules lookup and edge count after DELETE): {"rule": null, "owner_edges": 0}

4.3.14 Request 14: Delete saved search

```
DELETE http://localhost:8000/api/v1/saved-searches/265750
Accept: application/json
Content-Type: application/json
X-CSRFToken: <csrftoken cookie value, hidden>
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 204 (expected 204), server time 6.64 ms
Empty body (no content).
```

Database operation: REMOVE from `saved_searches`.

Direct read-back from ArangoDB (saved_searches lookup after DELETE): null

4.3.15 Request 15: Verify deletion (expect 404)

```
GET http://localhost:8000/api/v1/saved-searches/265750
Accept: application/json
Cookie: sessionid=<hidden>; csrftoken=<hidden>
```

```
HTTP 404 (expected 404), server time 7.91 ms
{
  "error": {
    "code": "not_found",
    "message": "Saved search not found.",
    "details": {},
    "correlation_id": "711f34eb1dd240b2ab66448089a1acbd"
  }
}
```

Database operation: Lookup returns no document, proving the DELETE persisted.

4.4 Findings and repairs made during the work

Table 10: Defects and environment issues found and resolved

Finding	Root cause	Repair and verification
Sign-up returned HTTP 500 when the password resembled the username (AttributeError: '_Candidate' object has no attribute '_meta')	apps/accounts/forms.py passed a plain helper object to Django's UserAttributeSimilarityValidator, which reads user._meta	The form now builds an unsaved user from get_user_model(); the form returns a normal validation error. Image rebuilt and redeployed; the request now returns HTTP 400 with 'The password is too similar to the username'. Regression test tests/unit/test_accounts_forms.py added.
Postman login failed with 500 'URL doesn't end in a slash'	The generated collection dropped the trailing slash of /accounts/login/	Path generation fixed; the collection now passes 15/15 under Newman.
Writes returned 403 'CSRF Failed' in the collection	The csrftoken cookie rotates at login and was read before rotation	The login test script stores the post-login token; all later requests send it in X-CSRFToken. Security protections were not weakened.

Finding	Root cause	Repair and verification
One regression test failed inside the container	The container inherits QW_DEBUG=true from the local .env, so a production-settings test hit the debug guard first	The test passes when QW_DEBUG is unset (environmental, not an application defect).

5. Test Cases and Results

The table lists the test cases executed for this review. Each was run against the live application and database; the Actual column states what was observed. The existing automated regression suite was run as well (Section 5.1).

Table 11: Executed test cases (36)

ID	Module	Scenario	Expected	Actual	Status
TC-01	API / Auth	GET /accounts/login/	HTTP 200	HTTP 200	Pass
TC-02	API / Auth	POST /accounts/login/	HTTP 302	HTTP 302	Pass
TC-03	API / Read	GET /api/v1/system/health	HTTP 200	HTTP 200	Pass
TC-04	API / Read	GET /api/v1/events	HTTP 200	HTTP 200	Pass
TC-05	API / Create	POST /api/v1/saved-searches	HTTP 201	HTTP 201	Pass
TC-06	API / Read	GET /api/v1/saved-searches/265750	HTTP 200	HTTP 200	Pass
TC-07	API / Update	PATCH /api/v1/saved-searches/265750	HTTP 200	HTTP 200	Pass
TC-08	API / Create	POST /api/v1/alert-rules	HTTP 201	HTTP 201	Pass
TC-09	API / Update	PATCH /api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e	HTTP 200	HTTP 200	Pass
TC-10	API / Read	GET /api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e/preview	HTTP 200	HTTP 200	Pass
TC-11	API / Create	POST /api/v1/research/snapshots	HTTP 201	HTTP 201	Pass
TC-12	API / Delete	DELETE /api/v1/research/snapshots/e748993f062e4ae6b38171f43fba1e6b	HTTP 204	HTTP 204	Pass
TC-13	API / Delete	DELETE /api/v1/alert-rules/bdcd462b4b9443768aed4a8f0fea109e	HTTP 204	HTTP 204	Pass
TC-14	API / Delete	DELETE /api/v1/saved-searches/265750	HTTP 204	HTTP 204	Pass
TC-15	API / Read	GET /api/v1/saved-searches/265750	HTTP 404	HTTP 404	Pass
TC-16	AQL	Run Q1 analytics_avg_magnitude_by_region	Executes, returns rows, uses an index where expected	20 row(s) in 6.98 ms	Pass
TC-17	AQL	Run Q2 analytics_dashboard_summary	Executes, returns rows, uses an index where expected	1 row(s) in 32.24 ms	Pass
TC-18	AQL	Run Q3 analytics_network_stats	Executes, returns rows, uses an index where expected	15 row(s) in 6.83 ms	Pass
TC-19	AQL	Run Q4 analytics_magnitude_changes	Executes, returns rows, uses an index where expected	10 row(s) in 3.15 ms	Pass
TC-20	AQL	Run Q5 analytics_ingestion_latency	Executes, returns rows, uses an index where expected	1 row(s) in 3.29 ms	Pass
TC-21	AQL	Run Q6 analytics_significant_by_week	Executes, returns rows, uses an index where expected	6 row(s) in 9.78 ms	Pass
TC-22	AQL	Run Q7 alerts_user_rules	Executes, returns rows, uses an index where expected	1 row(s) in 2.18 ms	Pass
TC-23	AQL	Run Q8 alerts_matched_events_graph	Executes, returns rows, uses an index where expected	1 row(s) in 1.22 ms	Pass

ID	Module	Scenario	Expected	Actual	Status
TC-24	AQL	Run Q9 event_graph_relationships	Executes, returns rows, uses an index where expected	1 row(s) in 31.85 ms	Pass
TC-25	AQL	Run Q10 events_in_viewport	Executes, returns rows, uses an index where expected	1 row(s) in 9.77 ms	Pass
TC-26	AQL	Run Q11 events_within_radius	Executes, returns rows, uses an index where expected	1 row(s) in 3.52 ms	Pass
TC-27	AQL	Run Q12 events_location_text	Executes, returns rows, uses an index where expected	1 row(s) in 4.11 ms	Pass
TC-28	AQL	Run Q13 ops_run_stats	Executes, returns rows, uses an index where expected	3 row(s) in 2.04 ms	Pass
TC-29	AQL	Run Q14 analytics_depth_histogram	Executes, returns rows, uses an index where expected	59 row(s) in 8.96 ms	Pass
TC-30	AQL	Run Q15 event_nearby_space_time	Executes, returns rows, uses an index where expected	7 row(s) in 1.56 ms	Pass
TC-31	Integrity	Insert schema-invalid event	Rejected by validator	Error 1620	Pass
TC-32	Integrity	Insert duplicate event identity	Unique constraint violation	Error 1210	Pass
TC-33	Transaction	Valid + invalid write in one transaction	Whole transaction rolled back	Aborted; probe document absent	Pass
TC-34	Transaction	Valid write in a stream transaction	Committed and visible	Committed	Pass
TC-35	Regression	Sign-up with password equal to username	HTTP 400 with validation message (was 500)	HTTP 400, message shown	Pass
TC-36	Collection	Postman/Newman run of the 15-request collection	0 failed assertions	15 requests, 35 assertions, 0 failed	Pass

5.1 Automated regression suite

The project's pytest suite (unit, contract, integration against a real ArangoDB test database, resilience and architecture-boundary tests; browser, performance and live-network tests excluded) was run in a container on 11 October 2026: **935 passed, 1 failed, 61 deselected**, line coverage 93 % over apps/. The single failure (test_production_requires_long_secret) is caused by the container inheriting QW_DEBUG=true from the local environment file; with that variable unset the test passes (1 passed). The suite was run after the sign-up repair and includes its new regression test.

6. Discussion and Limitations

- **Trigger and procedure equivalence.** ArangoDB offers no native triggers or stored procedures; equivalent guarantees are provided by server-side schema validation, unique and TTL indexes, stream transactions and application observers. This is a deliberate trade-off of the multi-model approach and is documented rather than hidden.
- **Live data.** Counts change every five minutes as the feed is ingested; tables report the values observed during the scripted run and screenshots may show slightly earlier values.

- **Backup scope.** The arangodump artifact was produced from an isolated test database; no restore test was carried out.
- **Repository state.** The sign-up repair and its test are present in the working tree and are included in the deployed image; they must be committed and pushed to appear on GitHub.
- **Screenshots.** Figures were captured by the project owner from the ArangoDB web interface; the textual results were produced independently by scripts, so the two sources corroborate each other.

References

ArangoDB Documentation, AQL, Indexing, ArangoSearch, Graphs and Transactions, version 3.12. https://docs.arangodb.com/3.12/

U.S. Geological Survey, Earthquake Hazards Program: GeoJSON Summary and Detail Feeds. https://earthquake.usgs.gov/earthquakes/feed/v1.0/geojson.php

Django Software Foundation, Django Documentation: Authentication, Sessions and CSRF protection. https://docs.djangoproject.com/

Django REST Framework, SessionAuthentication and Throttling. https://www.django-rest-framework.org/

Postman Learning Center, Collections, scripts and the Newman command-line runner. https://learning.postman.com/